PRUEBA TÉCNICA · ARQUITECTO DE INTEGRACIÓN

Arquitectura de integración para la modernización de un banco tradicional

Core tradicional y core digital, canales web y móvil, pagos, Open Finance, riesgos y prevención de fraudes

Preparado por: Joel Inca

Preparado para: Devsu

Versión: 1.0

Fecha: Octubre de 2026

Control del documento

Versión	Fecha	Descripción
0.1	Octubre 2026	Borrador inicial: contexto, supuestos y diagramas C4 de contexto y contenedores.
0.5	Octubre 2026	Componentes, modelo de eventos, estrategia multicore, IAM y seguridad.
1.0	Octubre 2026	Versión de entrega: HA/DR, observabilidad, gobierno, plan de migración y revisión general.

Contenido

1. Resumen de la propuesta 5
1.1 Cómo leer este documento 5
2. Contexto, alcance y supuestos 7
2.1 Situación de partida 7
2.2 Alcance 7
2.3 Supuestos de diseño 7
2.4 Requisitos no funcionales objetivo 8
3. Principios de arquitectura 9
4. Arquitectura propuesta: modelo C4 10
4.1 Visión general por capas 10
4.2 Nivel 1 · Diagrama de contexto 10
4.3 Nivel 2 · Diagrama de contenedores 12
4.4 Nivel 3 · Componentes del Payment Order Service 15
4.5 Nivel 3 · Componentes de la capa de integración multicore 17
4.6 Vista dinámica · Transferencia interbancaria desde la app 19
4.7 Principios de diseño aplicados a nivel de componentes 20
5. Alineación con BIAN 22
5.1 Cómo se usa BIAN en el diseño 22
5.2 Matriz de dominios 22
5.3 Matriz de sistemas de registro 23
5.4 Semántica de APIs alineada a BIAN 23
6. Patrones de integración y tecnologías 25
6.1 Patrones aplicados 25
6.2 Decisiones tecnológicas 26
7. Estrategia de APIs internas y externas 28
7.1 Clasificación de APIs 28
7.2 Estándares de diseño y mensajería 28
7.3 Ejemplos de API 28
7.4 Cuotas y protección de las APIs externas 30
8. Arquitectura orientada a eventos 31
8.1 Tipos de mensajes 31
8.2 Convenciones 31

8.3 Catálogo de eventos principales 31
8.4 Payloads de referencia 32
8.5 Versionado de eventos 32
8.6 Reintentos y dead-letter queues 33
8.7 Ordenamiento e idempotencia 33
8.8 Configuración de referencia de Kafka 33
9. Estrategia de integración multicore 35
9.1 El problema a resolver 35
9.2 Elementos de la estrategia y dónde están en el diseño 35
9.3 Reglas de ruteo 35
9.4 Transferencia entre cuentas de cores distintos 36
9.5 Diferencias de capacidad y horarios 36
10. Gestión de identidad y acceso 37
10.1 Dominios de identidad 37
10.2 Autenticación de clientes 37
10.3 Terceros Open Finance 37
10.4 Colaboradores 37
10.5 Servicio a servicio 38
10.6 Autorización en capas 38
11. Seguridad, cumplimiento normativo y protección de datos personales 39
11.1 Marco normativo considerado 39
11.2 Clasificación de datos 39
11.3 Elementos de la LOPDP y su presencia en la solución 39
11.4 Tabla de controles de seguridad por ubicación 40
11.5 Protección de datos personales en eventos, logs y entornos no productivos 41
12. Alta disponibilidad y recuperación ante desastres 42
12.1 Clasificación por criticidad 42
12.2 Topología de despliegue 43
12.3 Alta disponibilidad dentro de la región 44
12.4 Recuperación entre regiones 44
12.5 Escenarios de falla y comportamiento esperado 45
12.6 Pruebas de resiliencia 45
13. Gobierno de APIs y microservicios 46
13.1 Modelo organizativo 46
13.2 Ciclo de vida de una API o evento 46
13.3 Reglas verificadas automáticamente (fitness functions) 47
13.4 Métricas de gobierno 47
14. Plan de migración gradual 48
14.1 Enfoque 48
14.2 Fases 48
14.3 Migración de una cohorte 48

14.4 Riesgos de la migración 49
15. Observabilidad y trazabilidad 50
15.1 Modelo de observabilidad 50
15.2 Propagación del contexto de punta a punta 50
15.3 SLO y alertas 50
15.4 Auditoría y trazabilidad regulatoria 51
15.5 Tableros 51
16. Riesgos de la arquitectura y próximos pasos 52
Anexo A. Glosario 53

1. Resumen de la propuesta

El Banco necesita incorporar un core digital sin apagar el core tradicional, lanzar nuevos canales web y móvil, exponer APIs a terceros bajo un esquema de Open Finance y, al mismo tiempo, mantener integrados los sistemas de pagos, riesgos y prevención de fraudes. El reto principal no es conectar sistemas entre sí, sino hacerlo de forma que los dos cores puedan convivir durante varios años, que los canales no dependan de cuál core atiende a cada cliente y que la migración pueda avanzar por partes y revertirse si algo sale mal.

La propuesta organiza la solución en cinco capas: canales, borde (gateway, identidad y BFF), servicios de dominio alineados a BIAN, una capa de integración multicore y un backbone de eventos. Los puntos que sostienen el diseño son los siguientes:

- **Los cores nunca se exponen directamente.** Cada core tiene un adaptador anticorrupción (ACL) que traduce su modelo al modelo canónico basado en BIAN. Un componente de ruteo decide, cuenta por cuenta, qué core es el sistema de registro, y esa decisión es invisible para los canales y para los terceros.
- **Las consultas no viajan a los cores.** Saldos y movimientos se sirven desde un read model (Customer Position) alimentado por CDC desde el core tradicional y por eventos nativos del core digital. Esto protege al core tradicional, que es el componente con menor capacidad, y permite cumplir la latencia objetivo.
- **Los pagos se orquestan con una saga explícita.** La orden de pago tiene un dueño único (Payment Order Service), un estado persistido y compensaciones definidas. El canal recibe una respuesta 202 en menos de un segundo y la ejecución contra la plataforma de pagos ocurre de forma asíncrona.
- **Los eventos son el mecanismo de integración por defecto.** Kafka con Schema Registry, outbox transaccional, consumidores idempotentes, reintentos escalonados y DLQ. Las llamadas síncronas se limitan a las que el negocio no puede diferir (scoring de fraude, retención de fondos).
- **La seguridad se ubica en cada salto.** FAPI 2.0 y mTLS para terceros, tokens ligados al dispositivo (DPoP) para la app, mTLS con identidad SPIFFE entre servicios, autorización en capas y datos personales fuera de eventos y logs.
- **La migración es gradual y reversible.** Strangler fig por producto y por cohorte de clientes, con ejecución en paralelo (dual-run) y conciliación antes de cada cambio de sistema de registro.

La implementación de referencia usa AWS para la parte en nube y mantiene en el centro de datos del Banco el core tradicional y su adaptador. Se eligieron tecnologías con alternativas equivalentes (Kubernetes, Kafka, PostgreSQL, Keycloak, OpenTelemetry) para que el diseño pueda llevarse a Azure o a una nube privada sin rehacer la arquitectura; las decisiones de proveedor y sus alternativas se justifican en la sección 6.

1.1 Cómo leer este documento

La siguiente tabla relaciona cada tarea solicitada en el ejercicio con la sección donde se desarrolla, para facilitar la revisión.

Tabla 1. Trazabilidad entre las tareas del ejercicio y el documento

#	Tarea solicitada	Dónde se responde
1	Arquitectura de integración de alto nivel con BIAN	Secciones 4 (modelo C4) y 5 (alineación BIAN)
2	Patrones de integración y tecnologías justificados	Sección 6; aplicados en 4.4, 4.5, 8 y 9
3	Seguridad, cumplimiento normativo y LOPDP	Sección 11
4	Alta disponibilidad y recuperación ante desastres	Sección 12
5	Estrategia de integración multicore	Sección 9 y diagrama de componentes 4.5
6	Gestión de identidad y acceso	Sección 10
7	APIs internas y externas, estándares de mensajería	Secciones 7 y 8

#	Tarea solicitada	Dónde se responde
8	Gobierno de APIs y microservicios	Sección 13
9	Plan de migración gradual	Sección 14
10	Observabilidad y trazabilidad	Sección 15
11	Principios de diseño (SOLID, Clean Architecture, cohesión, acoplamiento)	Sección 4.7 y tabla de antipatrones evitados

2. Contexto, alcance y supuestos

2.1 Situación de partida

El Banco opera hoy con un core tradicional instalado en su centro de datos principal, con un centro de datos alterno en otra ciudad. Ese core es el sistema de registro de clientes, cuentas, préstamos y su contabilidad, y se integra con el resto de la organización mediante colas de mensajes, procedimientos almacenados y procesos batch nocturnos. Los canales actuales consumen esas interfaces casi de forma directa, por lo que cualquier cambio en el core se propaga a los canales.

La decisión de negocio es adquirir un core digital cloud-native para productos nuevos (por ejemplo, cuentas de ahorro 100 % digitales y créditos de bajo monto) y, con el tiempo, migrar hacia él la cartera existente. Mientras eso ocurre, ambos cores estarán activos y un mismo cliente puede tener productos en los dos.

2.2 Alcance

Dentro del alcance está la arquitectura de integración de los seis bloques solicitados: los dos cores, la banca web y móvil, la plataforma de servicios de pago, las APIs de Open Finance, el sistema de gestión de riesgos y el sistema de prevención de fraudes. Queda fuera del alcance la selección comercial del core digital, el rediseño de los productos financieros y la arquitectura interna de los sistemas de riesgo y fraude, que se tratan como productos existentes con interfaces conocidas.

2.3 Supuestos de diseño

Como el enunciado no detalla las interfaces ni la volumetría, se trabajó con los supuestos de la tabla siguiente. Cada uno indica qué decisión del diseño depende de él, de modo que si alguno no se cumple se sabe qué parte revisar.

Tabla 2. Supuestos y su impacto en el diseño

ID	Supuesto	Impacto en el diseño
S1	El core tradicional expone operaciones transaccionales por IBM MQ (request/reply) y su journal de base de datos es accesible para captura de cambios. Soporta alrededor de 150 TPS sostenidos.	Adaptador on-premise con bulkhead y límite de concurrencia; consultas resueltas desde read model y no desde el core.
S2	El core digital es un SaaS con APIs REST, webhooks o stream de eventos y conectividad privada (PrivateLink o equivalente).	ACL ligera en la nube; eventos nativos normalizados al modelo canónico.
S3	La plataforma de pagos incluye el switch transaccional, la conexión al Sistema de Pagos Interbancarios del BCE, redes de tarjetas y recaudaciones.	Un solo servicio (Payment Execution) absorbe el formato de cada rail; internamente se usa ISO 20022.
S4	Riesgos y fraudes son productos existentes con API de consulta en línea y capacidad de recibir eventos o archivos.	Servicios ACL propios (Fraud Screening, Limits) para no acoplar el dominio de pagos a esos productos.
S5	Volumetría de referencia: 1,5 millones de clientes digitales, 4 millones de transacciones diarias y picos de 600 TPS en canales en días de pago.	Dimensionamiento de particiones de Kafka, réplicas y límites de tasa.
S6	Rige la Ley Orgánica de Protección de Datos Personales del Ecuador (LOPDP) y su reglamento, la normativa de la Superintendencia de Bancos y PCI DSS para datos de tarjetas. El marco local de Open Finance está en construcción.	Las APIs externas se diseñan sobre estándares internacionales (FAPI 2.0, OAuth 2.0 RAR) para adaptarse a la norma local cuando se publique.
S7	El Banco ya mantiene replicación del core tradicional entre su centro de datos principal y el alterno.	La estrategia de DR reutiliza esa replicación; el adaptador sigue al core activo.

2.4 Requisitos no funcionales objetivo

Tabla 3. Requisitos no funcionales de referencia

Atributo	Objetivo	Cómo lo soporta el diseño
Disponibilidad de canales digitales	99,95 % mensual	Despliegue en tres zonas, degradación controlada ante caída de un core (sección 12).
Latencia de consulta de saldos	p95 < 300 ms en servidor	Read model en Redis/PostgreSQL; sin llamada al core.
Latencia de aceptación de transferencia	p95 < 800 ms en servidor	Saga con respuesta 202 tras retención de fondos; presupuesto de latencia en 4.6.
Frescura del read model	p99 < 2 s desde el posteo en el core	CDC por logs y eventos nativos; métrica de lag monitoreada.
RPO / RTO de servicios críticos	RPO ≤ 1 min / RTO ≤ 30 min ante pérdida de región	Warm standby en segunda región; replicación de base de datos y eventos.
Trazabilidad	100 % de operaciones monetarias con traza de punta a punta y registro de auditoría	OpenTelemetry, correlación por paymentOrderId, bitácora WORM.

3. Principios de arquitectura

Los principios no son declaraciones generales: cada uno se tradujo en una regla que se puede verificar en el diseño o en el pipeline de entrega. Se citan a lo largo del documento con su identificador.

Tabla 4. Principios de arquitectura y su verificación

ID	Principio	Qué significa en esta solución	Cómo se verifica
P1	Dominio primero	Los límites de los servicios siguen los service domains de BIAN y no la estructura de los cores ni de los canales.	Matriz de dominios (5.2); cada servicio declara su service domain en el catálogo.
P2	Un solo dueño por dato	Cada entidad tiene un único sistema de registro en cada momento. Los demás tienen copias derivadas, de solo lectura.	Matriz de sistemas de registro (5.3); credenciales de BD emitidas por servicio.
P3	Asíncrono por defecto	Se usa comunicación síncrona solo cuando la respuesta condiciona lo que el cliente ve en ese momento. La cadena síncrona no supera tres saltos internos.	Análisis de trazas en preproducción con alerta si la profundidad supera 3.
P4	El core no se expone	Ningún canal, tercero o servicio de dominio conoce el protocolo ni el modelo de un core.	Políticas de red y de mesh: solo los adaptadores llegan a los cores.
P5	Contratos explícitos	Toda API y todo evento tiene un contrato versionado (OpenAPI, AsyncAPI, Avro) antes de implementarse.	Linting y pruebas de contrato en el pipeline; compatibilidad en Schema Registry.
P6	Confianza cero	Cada llamada se autentica y autoriza, también dentro de la red del Banco.	mTLS obligatorio en el mesh; políticas deny-by-default.
P7	Diseñar para el fallo	Cada dependencia tiene timeout, estrategia de reintento y comportamiento degradado definido.	Tabla de escenarios de falla (12.5) y pruebas de caos.
P8	Portabilidad razonable	Se prefieren tecnologías con estándar abierto o equivalente en otra nube; los servicios administrados se usan cuando reducen operación sin crear dependencia difícil de revertir.	ADR con alternativa y costo de salida (6.2).
P9	Observable desde el diseño	La traza, las métricas y la auditoría forman parte del contrato del servicio, no se agregan después.	Checklist de producción y scorecard en el catálogo.

4. Arquitectura propuesta: modelo C4

4.1 Visión general por capas

Antes de entrar en los diagramas conviene explicar la lógica de capas, porque se repite en todos los niveles del modelo C4:

- **Canales.** Banca web (SPA) y banca móvil. No contienen reglas de negocio más allá de validaciones de formulario.
- **Borde.** CDN con WAF, API Gateway, Servidor de Identidad y un BFF por tipo de canal. El gateway aplica políticas transversales (token, cuotas, enrutamiento, versión) y nada más; la composición de pantallas vive en los BFF y las reglas de negocio en los servicios de dominio.
- **Servicios de dominio.** Un microservicio por service domain de BIAN que el Banco necesita gestionar con lógica propia: Party, Customer Position, Payment Order, Payment Execution, Consent, Fraud Screening, Limits y Notification. Cada uno es dueño exclusivo de su base de datos.
- **Integración multicore.** Core Integration Service (ruteo y ACL del core digital), Legacy Core Adapter (ACL del core tradicional, desplegado junto a él), CDC Connector y Reconciliation Service.
- **Backbone de eventos.** Kafka con Schema Registry. Es el medio por el que los dominios se enteran de lo que pasa en otros dominios sin llamarse entre sí.
- **Plataforma transversal.** Observabilidad, auditoría, secretos y llaves. La usan todos los contenedores, por eso en el diagrama de contenedores se muestra sin flechas para no saturarlo.

La regla que más influye en la forma del diseño es P3: un comando del cliente atraviesa como máximo tres saltos síncronos internos (BFF → servicio de dominio → servicio de integración → adaptador) y una consulta atraviesa uno (BFF → read model). Todo lo demás se propaga por eventos.

4.2 Nivel 1 · Diagrama de contexto

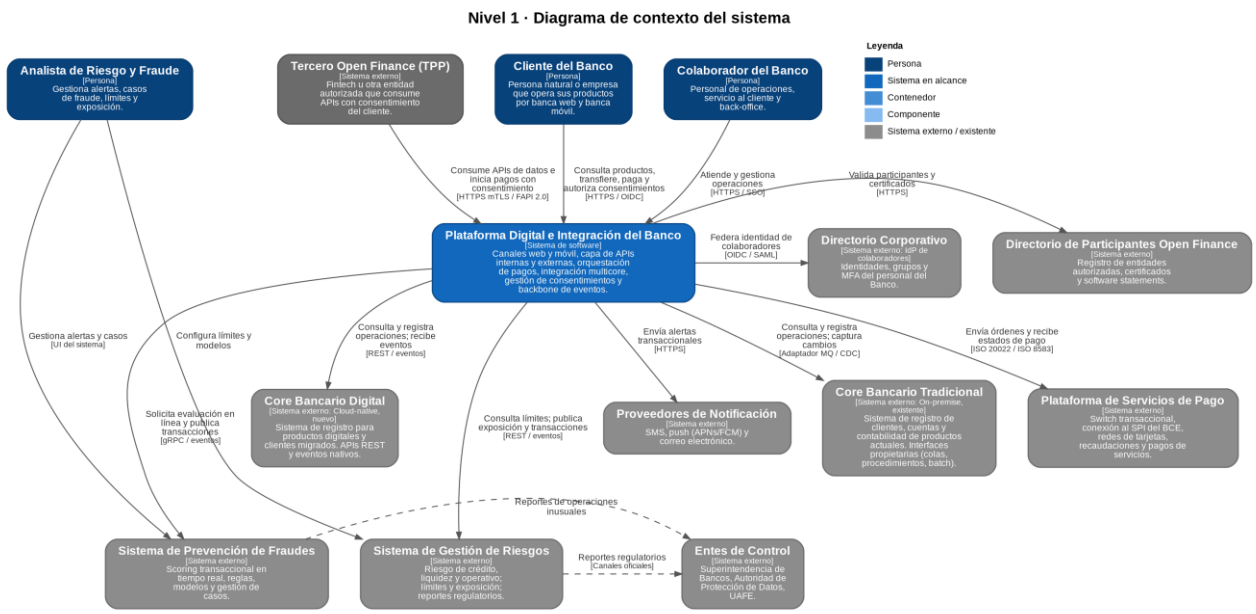


Figura 1. Diagrama de contexto del sistema (C4 nivel 1)

El sistema en alcance es la **Plataforma Digital e Integración del Banco**. Se modela como un solo sistema porque es lo que el Banco construye y gobierna; los dos cores, la plataforma de pagos y los sistemas de riesgo y fraude se muestran como sistemas externos porque son productos existentes o adquiridos cuyo interior no se rediseña.

Algunas relaciones merecen explicación:

- El **Tercero Open Finance** no es una persona sino un sistema, pero se ubica junto a los actores porque, desde el punto de vista del Banco, es un consumidor externo con identidad propia (certificado y registro en el directorio de participantes).
- El **Analista de Riesgo y Fraude** trabaja sobre las herramientas de esos sistemas. La plataforma no replica esas pantallas; solo les entrega eventos y les consulta decisiones.
- Los **reportes a entes de control** salen de los sistemas especializados. La plataforma aporta los datos (eventos de transacciones y exposición) y la bitácora de auditoría, pero no genera reportes regulatorios por su cuenta.
- La relación con el **Directorio de Participantes** permite validar que un tercero sigue autorizado antes de emitirle tokens; si el directorio revoca a un participante, el Banco deja de aceptarlo sin intervención manual.

4.3 Nivel 2 · Diagrama de contenedores

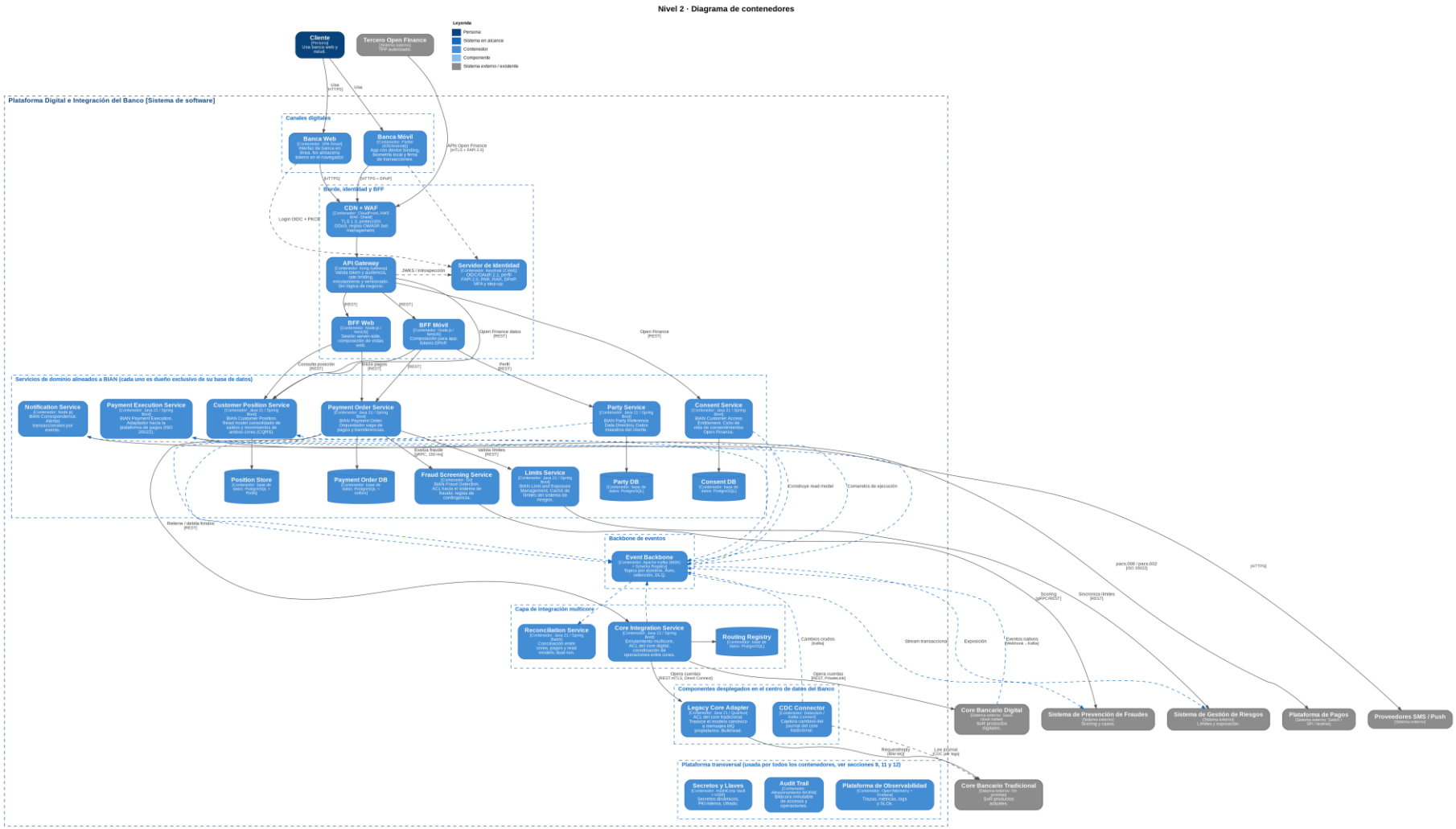


Figura 2. Diagrama de contenedores (C4 nivel 2). Líneas continuas: llamadas síncronas. Líneas punteadas azules: publicación o consumo de eventos.

El diagrama de contenedores muestra las unidades desplegables y la forma en que se comunican. Para leerlo conviene seguir dos recorridos: el de un comando (iniciar una transferencia) por las flechas continuas, y el de un hecho (se registró un movimiento en el core) por las flechas punteadas que pasan por el Event Backbone. La siguiente tabla detalla la responsabilidad de cada contenedor y, sobre todo, qué datos posee, porque ahí es donde suelen aparecer ambigüedades.

Tabla 5. Responsabilidad y propiedad de datos por contenedor

Contenedor	Responsabilidad única	Datos que posee	Se comunica con
API Gateway (Kong)	Políticas transversales de entrada: validación de token, audiencia y DPoP/mTLS, cuotas, enrutamiento por versión. No transforma payloads ni decide reglas de negocio.	Ninguno (configuración declarativa en Git).	BFF, Consent Service, Customer Position (Open Finance), Servidor de Identidad (JWKS).
Servidor de Identidad (Keycloak)	Autenticación de clientes, terceros y colaboradores federados; emisión de tokens.	Credenciales, dispositivos vinculados, clientes OAuth registrados.	Canales, gateway, Directorio de Participantes, IdP corporativo.
BFF Web / BFF Móvil	Adaptar las APIs de dominio a cada experiencia: agregación de vistas, sesión server-side (web), paginación de pantalla.	Sesión de usuario (web), sin datos de negocio.	Servicios de dominio vía REST.
Party Service	Datos maestros del cliente (BIAN Party Reference Data Directory). Golden record tras la migración.	Datos de identificación y contacto del cliente.	Publica party.*; consume cambios del core tradicional en la fase de transición.
Customer Position Service	Vista consolidada de saldos y movimientos de ambos cores (CQRS, lado de lectura).	Read model derivado; nunca es sistema de registro.	Consume accounts.*; atiende BFF y Open Finance.
Payment Order Service	Recibir, validar y orquestar órdenes de pago hasta su estado final.	Órdenes de pago, estado de saga, outbox.	Fraud Screening, Limits, Core Integration (síncrono); eventos payment-order.*
Payment Execution Service	Ejecutar la orden en el rail correspondiente y traducir a su formato (ISO 20022 / ISO 8583).	Transacciones de ejecución y su estado en el rail.	Plataforma de pagos; eventos payment-execution.*
Consent Service	Ciclo de vida de consentimientos Open Finance: creación, autorización, consulta, revocación y expiración.	Consentimientos y su historial.	Gateway, Servidor de Identidad; publica openfinance.consent.*
Fraud Screening Service	ACL hacia el sistema de fraude; reglas de contingencia si ese sistema no responde a tiempo.	Reglas de contingencia y caché de listas.	Sistema de prevención de fraudes (gRPC/REST).
Limits Service	ACL hacia el sistema de riesgos para límites transaccionales y de exposición.	Copia cacheada de límites (no es sistema de registro).	Sistema de gestión de riesgos.
Notification Service	Alertas transaccionales y de seguridad al cliente (BIAN Correspondence).	Preferencias de notificación y bitácora de envíos.	Consume eventos; proveedores SMS/push.
Core Integration Service	Resolver qué core atiende cada cuenta y ejecutar la operación a través del puerto correspondiente.	Routing Registry (reglas y asignación cuenta → core).	Core digital (REST), Legacy Core Adapter (REST mTLS).
Legacy Core Adapter	Traducir el modelo canónico al protocolo del core tradicional y protegerlo de sobrecarga.	Ninguno persistente (catálogo de mapeos versionado en Git).	Core tradicional (IBM MQ).
CDC Connector	Capturar los cambios confirmados en el core tradicional sin consultarlo.	Offsets de lectura.	Journal del core; Kafka.
Reconciliation Service	Conciliar posiciones entre cores, órdenes de pago, ejecuciones y read models.	Resultados y diferencias de conciliación.	Consume eventos; consulta lotes de cierre de los cores.

Hay tres decisiones del diagrama que vale la pena justificar:

- **Fraud Screening y Limits son contenedores propios y no librerías dentro de Payment Order.** Ambos encapsulan a un producto externo que puede cambiar. Si mañana se reemplaza el sistema de fraude, solo cambia el adaptador interno de Fraud Screening; Payment Order sigue hablando con el mismo contrato.

Además, Open Finance y otros flujos futuros (pagos de servicios, avances de tarjeta) reutilizan esos mismos contratos.

- **El Legacy Core Adapter se despliega en el centro de datos y no en la nube.** El core tradicional usa request/reply sobre MQ con timeouts cortos; poner el adaptador al lado del core evita que cada mensaje cruce el enlace dedicado, y deja un único cruce nube–centro de datos por operación, sobre HTTP/2 con mTLS.
- **Customer Position no llama a los cores.** Si un core no está disponible, los clientes siguen viendo su posición con la marca de hora del último dato recibido. Esto convierte la caída de un core en una degradación parcial y no en una caída de canales.

4.4 Nivel 3 · Componentes del Payment Order Service

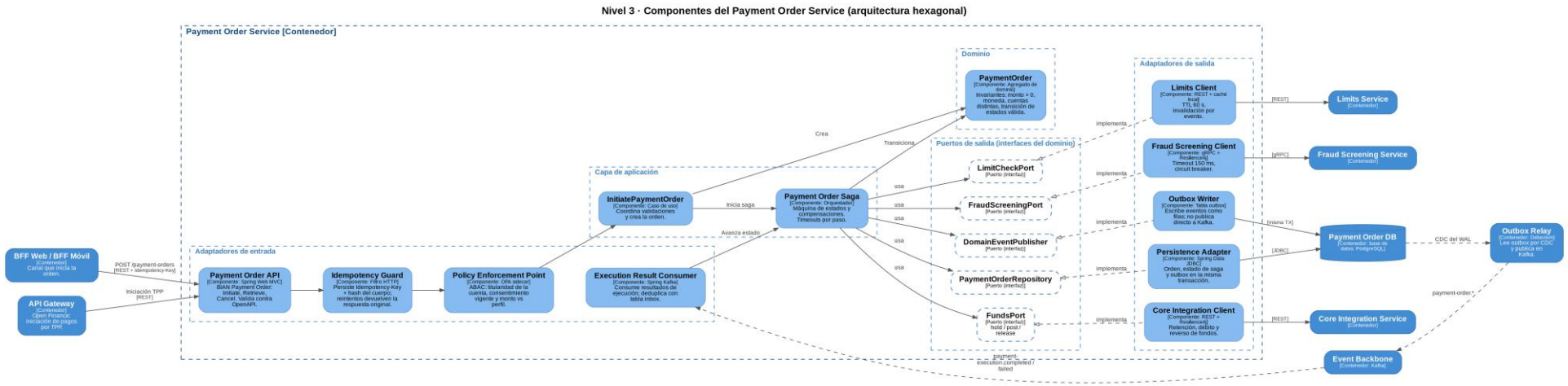


Figura 3. Componentes del Payment Order Service organizados en arquitectura hexagonal (C4 nivel 3)

Se eligió el Payment Order Service para el nivel de componentes porque concentra las decisiones más delicadas: idempotencia de cara al cliente, orquestación con compensaciones, consistencia entre la base de datos y los eventos, y llamadas síncronas con presupuesto de latencia. El diseño sigue arquitectura hexagonal: el dominio y la capa de aplicación no conocen gRPC, Kafka, PostgreSQL ni el core; solo conocen puertos.

Recorrido de una orden

1. La **Payment Order API** recibe la solicitud y la valida contra el contrato OpenAPI (tipos, formatos, montos con dos decimales, moneda ISO 4217).
2. El **Idempotency Guard** busca la cabecera Idempotency-Key. Si ya existe con el mismo hash de cuerpo, devuelve la respuesta almacenada; si existe con otro cuerpo, responde 422. Así un reintento del canal por pérdida de señal no genera un segundo pago.
3. El **Policy Enforcement Point** consulta al sidecar OPA si el sujeto del token es titular de la cuenta origen, si el consentimiento (cuando la orden viene de un tercero) cubre ese monto y ese beneficiario, y si el nivel de autenticación (acr) es suficiente para el monto.
4. El caso de uso **InitiatePaymentOrder** construye el agregado **PaymentOrder**, que valida sus invariantes, y arranca la **Payment Order Saga**.
5. La saga ejecuta los pasos síncronos (fraude, límites, retención de fondos) a través de los puertos, persiste el estado junto con los eventos en la tabla outbox en una sola transacción y responde 202.
6. El **Outbox Relay** (Debezium) publica los eventos en Kafka. El **Execution Result Consumer** recibe el resultado del rail, descarta duplicados con la tabla inbox y hace avanzar la saga hasta su estado final.

Estados de la saga y compensaciones

La saga es orquestada, no coreografiada. En pagos se necesita un lugar único que sepa en qué estado está cada orden, que responda consultas del cliente y del área de operaciones, y que decida compensaciones con un orden conocido. La coreografía se reserva para efectos secundarios que no requieren coordinación (notificaciones, read models, analítica).

Tabla 6. Pasos de la saga de pago interbancario

Estado alcanzado	Acción	Si falla	Tipo de recuperación
RECEIVED	Validación de contrato, idempotencia y autorización ABAC.	Rechazo inmediato (4xx). No se crea orden.	No aplica.
SCREENED	Scoring de fraude (timeout 150 ms).	DECLINE → REJECTED. REVIEW → PENDING_REVIEW hasta decisión del analista o expiración. Timeout → reglas de contingencia.	Sin compensación: aún no hay efectos.
LIMIT_CHECKED	Validación de límite diario y por transacción.	REJECTED con código de límite excedido.	Sin compensación.
FUNDS_RESERVED	Retención (hold) en la cuenta origen a través de Core Integration.	Fondos insuficientes → REJECTED. Core no disponible → REJECTED con código de reintento.	Sin compensación.
SUBMITTED	Comando de ejecución publicado; Payment Execution envía pacs.008.	pacs.002 RJCT → liberar hold → REJECTED.	Compensación: release del hold.
IN_INQUIRY	Sin respuesta del rail dentro del tiempo de saga.	Se consulta el estado al rail (pacs.028) antes de liberar fondos.	Evita liberar fondos de un pago que sí salió.
SETTLED → COMPLETED	Débito definitivo (conversión del hold) y evento de cierre.	Si el core no responde, se reintenta con la misma clave de operación hasta confirmar.	Recuperación hacia adelante: el dinero ya salió, no se compensa.

> El estado IN_INQUIRY es una decisión deliberada. El error más costoso en una saga de pagos es compensar (liberar fondos) cuando el rail sí procesó el pago, porque genera un faltante real. Ante la duda, la saga consulta antes de compensar.

4.5 Nivel 3 · Componentes de la capa de integración multicore

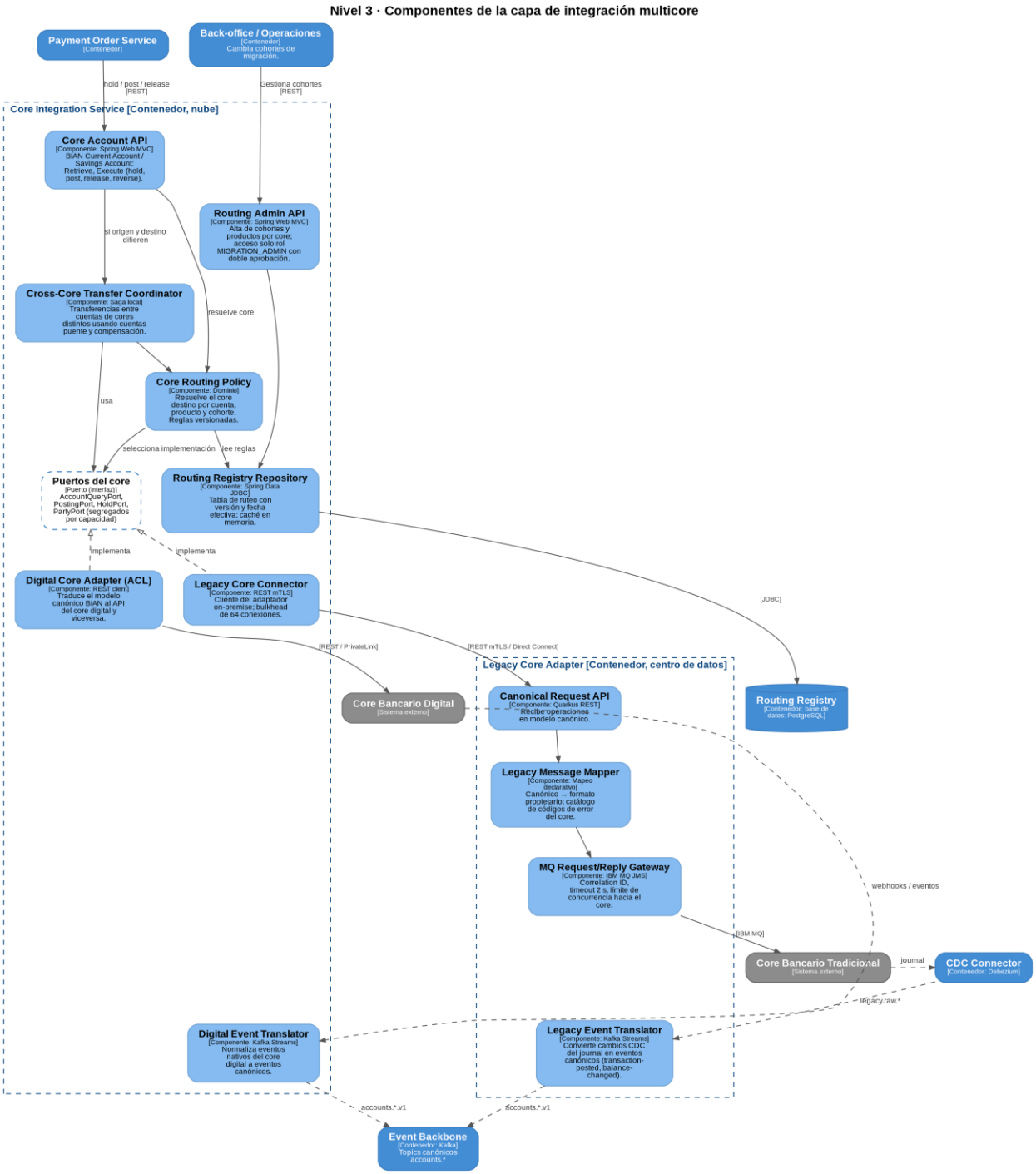


Figura 4. Componentes del Core Integration Service y del Legacy Core Adapter (C4 nivel 3)

Este diagrama es donde se materializa la estrategia multicore (detallada en la sección 9). Los componentes clave son:

- **Core Account API** expone las operaciones de cuenta con semántica BIAN (Current Account y Savings Account): consultar, retener, debitar, acreditar, liberar y reversar. Es el único punto por el que los servicios de dominio operan sobre cuentas.
- **Core Routing Policy** decide el core destino. No contiene reglas en código: las lee del Routing Registry, que guarda asignaciones por producto, por cohorte de migración y por cuenta, cada una con versión y fecha efectiva. El cambio de core de un cliente es un cambio de datos auditado, no un despliegue.

- **Puertos del core** están segregados por capacidad (AccountQueryPort, PostingPort, HoldPort, PartyPort). Un core que no soporte retenciones, por ejemplo, simplemente no implementa HoldPort y la política de ruteo lo sabe.
- **Digital Core Adapter** y **Legacy Core Connector** son las dos implementaciones de esos puertos. El segundo no habla MQ: delega en el Legacy Core Adapter on-premise, que contiene el mapeo al formato propietario y el control de concurrencia.
- **Cross-Core Transfer Coordinator** resuelve las transferencias en las que la cuenta origen está en un core y la destino en otro, usando cuentas puente contables en cada core y una saga local con compensación.
- **Legacy Event Translator** y **Digital Event Translator** convierten los cambios de cada core en los mismos eventos canónicos (accounts.transaction.posted.v1, accounts.balance.changed.v1). Los consumidores no distinguen de qué core vino un movimiento salvo por un atributo informativo.

4.6 Vista dinámica · Transferencia interbancaria desde la app

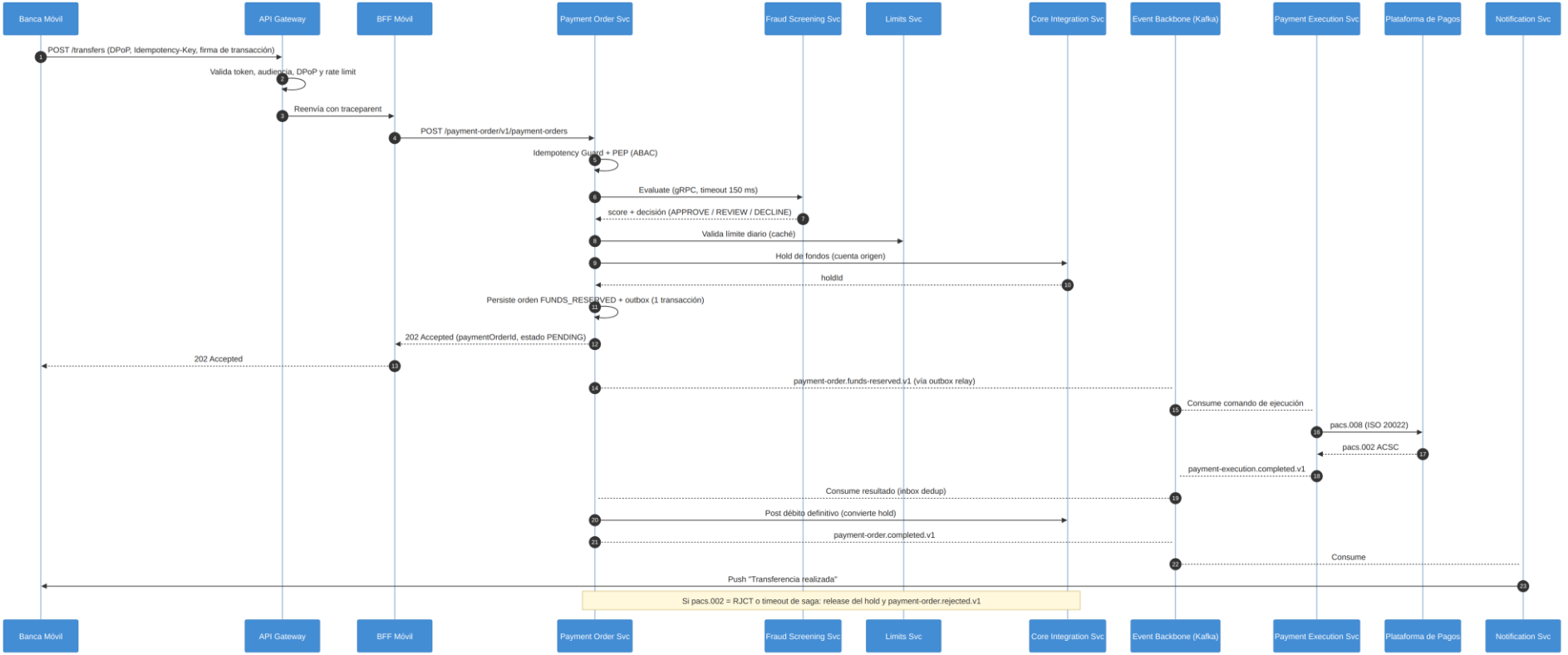


Figura 5. Vista dinámica de una transferencia interbancaria. Flechas continuas: llamadas síncronas. Flechas punteadas: respuestas o eventos.

La vista dinámica complementa los diagramas estáticos y muestra dónde termina la parte síncrona (paso 13, respuesta 202 al cliente) y dónde empieza la asíncrona. El cliente ve inmediatamente su orden en estado pendiente con los fondos ya retenidos, y recibe la confirmación por push cuando el rail responde, lo que en condiciones normales ocurre en pocos segundos.

El presupuesto de latencia de la parte síncrona se calculó para comprobar que el objetivo de p95 menor a 800 ms es alcanzable con un core tradicional lento:

Tabla 7. Presupuesto de latencia de la aceptación de una transferencia (p95, en servidor)

Tramo	Presupuesto	Mecanismo de control
CDN/WAF + API Gateway	20 ms	Validación local de JWT con JWKS en caché; sin introspección remota.
BFF Móvil	15 ms	Sin agregación en este flujo; solo reenvío con contexto.
Payment Order: validación, idempotencia, PEP	25 ms	OPA como sidecar (decisión local).
Fraud Screening	150 ms (timeout)	gRPC; al vencer, reglas de contingencia locales.
Limits	10 ms	Caché local con invalidación por evento.
Retención de fondos (Core Integration → adaptador → core tradicional)	450 ms	Un cruce nube–DC (~70 ms RTT) + respuesta del core. Timeout 800 ms.
Persistencia de orden + outbox	20 ms	Una transacción local en PostgreSQL.
Total estimado	≈ 690 ms	Holgura para variabilidad de red.

Si la cuenta origen está en el core digital, el tramo de retención baja a unos 120 ms, porque la conexión es privada dentro de la nube. La migración de clientes al core digital mejora directamente la experiencia, lo que también sirve como argumento de negocio para el plan de la sección 14.

4.7 Principios de diseño aplicados a nivel de componentes

El enunciado pide evidenciar los principios de diseño en los límites de servicios, APIs, eventos y adaptadores. La siguiente tabla los relaciona con elementos concretos del modelo, no con buenas intenciones.

Tabla 8. Principios SOLID y de Clean Architecture en la solución

Principio	Dónde se refleja
Responsabilidad única (SRP)	Cada microservicio corresponde a un service domain de BIAN y tiene un solo motivo de cambio. Payment Order cambia si cambia el proceso de pago; Payment Execution cambia si cambia un rail. Por eso son dos servicios y no uno. En el nivel de componentes, Idempotency Guard, PEP y Saga son componentes separados porque cambian por razones distintas (protocolo, seguridad, negocio).
Abierto/cerrado (OCP)	Agregar un tercer core (por ejemplo, un core de tarjetas) implica escribir un adaptador nuevo que implemente los puertos del core y registrar reglas en el Routing Registry. No se modifican Core Routing Policy, Payment Order ni los canales. Lo mismo aplica a un nuevo rail de pagos en Payment Execution.
Sustitución de Liskov (LSP)	Los dos adaptadores de core deben comportarse igual frente a los mismos puertos: mismos códigos de error canónicos, misma semántica de idempotencia por clave de operación. Se verifica con una suite de pruebas de contrato que corre contra ambos adaptadores (y contra un simulador de cada core) en el pipeline.
Segregación de interfaces (ISP)	Los puertos del core están separados por capacidad en lugar de una interfaz CoreBankingService con cuarenta métodos. Hacia afuera, cada tipo de consumidor tiene su propia API: los terceros no ven la API de dominio, ven la API Open Finance; la app no ve la API de dominio, ve su BFF.
Inversión de dependencias (DIP)	El dominio define los puertos (FraudScreeningPort, FundsPort…) y los adaptadores los implementan. La dependencia de compilación va del adaptador al dominio, nunca al revés. Esto permite probar la saga completa con dobles de prueba, sin Kafka ni core.

Principio	Dónde se refleja
Separación de responsabilidades	Gateway: políticas transversales. BFF: experiencia. Dominio: reglas. Integración: traducción y ruteo. Plataforma: infraestructura. Ninguna capa invade a otra; en particular, el gateway no contiene lógica de negocio (regla verificada en su configuración, sección 13).
Clean Architecture	Dentro de cada servicio, la regla de dependencias apunta hacia el dominio: adaptadores de entrada → aplicación → dominio ← puertos ← adaptadores de salida. El modelo canónico BIAN vive en el dominio; los modelos de los cores viven solo en sus ACL.

Cohesión, acoplamiento, testabilidad y evolución independiente

- **Cohesión alta:** los datos y las reglas que cambian juntos están en el mismo servicio. El estado de una orden y sus transiciones están en Payment Order; nadie más los modifica.
- **Acoplamiento bajo:** los servicios se conocen por contratos, no por implementaciones. El acoplamiento temporal se reduce con eventos: si Notification Service está caído, los pagos siguen funcionando y las notificaciones salen cuando se recupera.
- **Contratos explícitos:** OpenAPI 3.1 para APIs, AsyncAPI 3.0 y Avro para eventos, todos versionados en un repositorio de contratos con revisión obligatoria.
- **Testabilidad:** pruebas unitarias del dominio sin infraestructura; pruebas de contrato dirigidas por el consumidor (Pact) entre BFF y servicios; simuladores de ambos cores para pruebas de integración; validación de compatibilidad de esquemas en CI.
- **Evolución independiente:** cada servicio tiene su propio pipeline y su propia base de datos. Un cambio compatible se despliega sin coordinar con otros equipos; uno incompatible sigue la política de versionado (7.2 y 8.5).

Antipatrones evitados

Tabla 9. Antipatrones señalados en el enunciado y cómo los evita el diseño

Antipatrón	Cómo se evita
Microservicios sin responsabilidad clara	Cada servicio tiene un service domain BIAN y una fila en la matriz de dominios con su dueño. No existen servicios "utilitarios" genéricos.
Propiedad ambigua de datos	Matriz de sistemas de registro por entidad y por fase de migración (5.3). Los read models se declaran explícitamente como derivados.
Base de datos compartida	Una base por servicio con credenciales dinámicas emitidas por Vault para la cuenta de servicio dueña. La integración entre servicios es por API o evento.
Cadenas síncronas excesivas	Regla P3 (máximo tres saltos) verificada con trazas. Consultas por read model; efectos secundarios por eventos.
Lógica de negocio en el API Gateway	El gateway solo usa plugins de una lista permitida (autenticación, cuotas, CORS, enrutamiento). Validaciones de negocio y de consentimiento están en los servicios.
Referencias superficiales a BIAN	BIAN define los límites de servicio, el modelo canónico, la semántica de las operaciones y la nomenclatura de eventos (sección 5).
Dependencia de proveedor sin justificación	Cada decisión tecnológica tiene alternativa y costo de salida documentados (6.2).
Controles de seguridad como lista genérica	Tabla de controles con ubicación exacta en la arquitectura (11.4).

5. Alineación con BIAN

5.1 Cómo se usa BIAN en el diseño

BIAN se usa en cuatro niveles del diseño, y en cada uno resuelve un problema concreto:

- **Límites de servicio.** Los service domains son la guía para decidir qué es un microservicio. Evita la tentación de partir por tablas del core (un servicio por tabla) o por pantallas del canal (un servicio por pantalla), que son los dos errores más comunes en modernizaciones bancarias.
- **Modelo canónico.** Los objetos de negocio que viajan entre servicios (cuenta, orden de pago, transacción, parte) se modelan a partir de los business objects de BIAN. Ese es el idioma que los ACL usan para traducir desde y hacia cada core, y es lo que permite que dos cores con modelos distintos se vean iguales hacia arriba.
- **Semántica de operaciones.** Las operaciones de las APIs de dominio siguen los action terms de BIAN (Initiate, Retrieve, Update, Execute, Control, Evaluate, Notify), traducidos a convenciones REST (tabla de 5.4).
- **Nomenclatura de eventos.** Los eventos se nombran por service domain y por hecho de negocio, lo que hace que el catálogo de eventos se pueda leer con el mismo mapa que el catálogo de servicios.

No se pretende implementar todos los service domains de BIAN. El banco solo construye servicios propios donde necesita lógica o datos propios; el resto de capacidades las cubren los cores y los productos existentes, y BIAN sirve para describir qué capacidad aporta cada uno.

5.2 Matriz de dominios

Tabla 10. Matriz de dominios de negocio, service domains BIAN y servicios

Dominio de negocio	Service domain BIAN	Implementado por	Responsabilidad en la solución	Equipo dueño
Clientes	Party Reference Data Directory	Party Service	Golden record de datos de identificación y contacto; resolución de identidad entre cores.	Clientes y onboarding
Clientes	Customer Position	Customer Position Service	Posición consolidada de productos, saldos y movimientos para canales y Open Finance.	Canales digitales
Cuentas	Current Account / Savings Account	Core tradicional y core digital (vía Core Integration Service)	Ledger de cuentas, retenciones, débitos y créditos. El Banco no reimplementa estas capacidades.	Integración core
Pagos	Payment Order	Payment Order Service	Intención de pago del cliente, validaciones, orquestación y estado.	Pagos
Pagos	Payment Execution	Payment Execution Service	Ejecución técnica en cada rail y traducción de formatos.	Pagos
Pagos	Payment Rail Operations	Plataforma de servicios de pago (existente)	Operación del switch, conexión al SPI, redes de tarjetas.	Operaciones de pagos
Open Finance	Customer Access Entitlement	Consent Service	Consentimientos de acceso de terceros: alcance, vigencia y revocación.	Open Finance
Seguridad de canal	Party Authentication	Servidor de Identidad	Autenticación del cliente, factores y dispositivos.	Seguridad / IAM
Fraude	Fraud Detection	Fraud Screening Service + sistema de fraudes	Evaluación en línea; contingencia cuando el sistema no responde.	Prevención de fraude
Riesgos	Limit and Exposure Management	Limits Service + sistema de riesgos	Límites transaccionales y de exposición disponibles en línea.	Riesgos
Riesgos	Customer Credit Rating	Sistema de riesgos (existente)	Calificación usada en originación de créditos digitales.	Riesgos
Comunicación	Correspondence	Notification Service	Alertas transaccionales y de seguridad.	Canales digitales

Dominio de negocio	Service domain BIAN	Implementado por	Responsabilidad en la solución	Equipo dueño
Operaciones	Customer Event History / Channel Activity History	Plataforma de observabilidad y Audit Trail	Historial de actividad del cliente para soporte, fraude y auditoría.	Plataforma

5.3 Matriz de sistemas de registro

La matriz de sistemas de registro (SoR) responde quién manda sobre cada dato. Como el banco atraviesa una migración, se indica el SoR actual, el objetivo y cómo se sincronizan las copias. Ninguna copia se edita: si un dato está mal, se corrige en su SoR y la corrección llega por evento.

Tabla 11. Sistemas de registro por entidad de datos

Entidad	SoR actual	SoR objetivo	Copias derivadas	Sincronización
Datos maestros del cliente	Core tradicional	Party Service (golden record)	Core digital (datos mínimos para operar), Customer Position	Fase 1-2: CDC del core → Party. Fase 3+: Party publica party.updated y ambos cores se actualizan por ACL.
Cuentas y saldos de productos actuales	Core tradicional	Core digital, por cohorte	Customer Position	CDC del journal; cambio de SoR por cohorte vía Routing Registry.
Cuentas de productos digitales	Core digital	Core digital	Customer Position	Eventos nativos normalizados por Digital Event Translator.
Movimientos contables de cuenta	Core que lleva la cuenta	Core que lleva la cuenta	Customer Position, sistemas de fraude y riesgo	accounts.transaction.posted.v1
Orden de pago y su estado	Payment Order Service	Payment Order Service	Customer Position (pendientes), Reconciliation	payment-order.*
Ejecución en el rail	Payment Execution Service	Payment Execution Service	Reconciliation	payment-execution.*; conciliación con archivos del rail.
Consentimientos Open Finance	Consent Service	Consent Service	Caché en Customer Position y Payment Order	openfinance.consent.*
Límites y exposición	Sistema de riesgos	Sistema de riesgos	Limits Service (caché)	Carga inicial + eventos de cambio de límite; TTL 60 s.
Casos y alertas de fraude	Sistema de fraudes	Sistema de fraudes	Ninguna	fraud.alert.raised.v1 para notificar al cliente.
Asignación cuenta → core	Core Integration Service	Core Integration Service	Caché en memoria del mismo servicio	core.routing.changed.v1
Identidad digital y dispositivos	Servidor de Identidad	Servidor de Identidad	Ninguna	—
Contabilidad general consolidada	Sistema contable existente	Sistema contable existente	—	Interfaces contables de cada core (sin cambios en esta iniciativa).

5.4 Semántica de APIs alineada a BIAN

Las APIs de dominio usan rutas REST orientadas a recursos y no la sintaxis literal de BIAN, que resulta poco natural para los equipos y para las herramientas. La correspondencia con los action terms se mantiene documentada en el contrato con la extensión x-bian-action, de forma que la trazabilidad con BIAN no se pierde.

Tabla 12. Correspondencia entre action terms de BIAN y operaciones REST

Action term	Operación REST	Ejemplo
Initiate	POST sobre la colección	POST /payment-order/v1/payment-orders
Retrieve	GET sobre el recurso o la colección	GET /payment-order/v1/payment-orders/{paymentOrderId}
Update	PATCH sobre el recurso	PATCH /party/v1/parties/{partyId}/contact-points
Execute	POST sobre una sub-acción	POST /core/v1/current-accounts/{accountId}/holds
Control	POST sobre una sub-acción de control	POST /payment-order/v1/payment-orders/{id}/cancellation
Evaluate	POST de evaluación sin efecto persistente	POST /fraud-screening/v1/evaluations
Notify	Evento publicado	payment-order.completed.v1

6. Patrones de integración y tecnologías

6.1 Patrones aplicados

Cada patrón se incluyó porque resuelve un problema identificable en este escenario. La tabla indica dónde está aplicado (referencia al diagrama), por qué, y qué costo introduce, porque ningún patrón es gratis.

Tabla 13. Patrones de integración: ubicación, justificación y costo

Patrón	Dónde	Por qué se aplica aquí	Costo o cuidado
API Gateway	Borde (fig. 2)	Punto único para aplicar seguridad, cuotas y versionado a canales y terceros.	Se limita a políticas transversales para no convertirse en un monolito de integración.
Backend for Frontend	BFF Web y BFF Móvil	La web y la app tienen necesidades distintas (sesión server-side vs. tokens DPoP, tamaño de payload). Evita que las APIs de dominio se deformen por necesidades de pantalla.	Dos componentes más que mantener; se controla con equipos de canal dueños de su BFF.
Anti-Corruption Layer	Legacy Core Adapter, Digital Core Adapter, Fraud Screening, Limits, Payment Execution	Evita que el modelo y las limitaciones de productos externos se filtren al dominio. Permite reemplazar un producto cambiando un solo componente.	Mapeos que hay que mantener y probar; se versionan y se prueban con simuladores.
Strangler Fig	Core Routing Policy + Routing Registry	Permite mover funcionalidad y clientes del core tradicional al digital sin un corte único.	Convivencia prolongada; exige conciliación continua.
Content-Based Router	Core Routing Policy	El destino de cada operación depende de datos (cuenta, producto, cohorte).	La tabla de ruteo pasa a ser un dato crítico: se audita y se versiona.
Saga orquestada	Payment Order Saga	Una transferencia toca core, fraude, riesgo y rail; no hay transacción distribuida posible. Se necesita estado explícito y compensaciones ordenadas.	Complejidad de estados; se mitiga con máquina de estados declarativa y pruebas por transición.
Transactional Outbox	Payment Order, Party, Consent, Payment Execution	Garantiza que el cambio de estado y su evento se confirmen juntos, sin doble escritura.	Requiere relay (Debezium) y limpieza de la tabla.
Idempotent Consumer (Inbox)	Todos los consumidores con efectos	Kafka entrega al menos una vez; los duplicados deben ser inocuos.	Tabla de eventos procesados con retención.
Change Data Capture	CDC Connector sobre el core tradicional	Obtener cambios del core sin modificarlo ni consultarlo. Es la única forma realista de tener eventos de un core que no los emite.	Los cambios llegan en modelo físico; el Legacy Event Translator los convierte a eventos de negocio.
CQRS / read model	Customer Position	Las consultas son el 85 % del tráfico de canales; separarlas de los comandos protege a los cores y baja la latencia.	Consistencia eventual (p99 < 2 s); se muestra la hora del dato al cliente.
Event-carried state transfer	Caché de consentimientos y límites	Los servicios tienen localmente lo que necesitan para decidir, sin llamada síncrona adicional.	Copias que hay que invalidar con eventos de cambio.
Request/Reply con Correlation Identifier	MQ Request/Reply Gateway	Es el protocolo nativo del core tradicional.	Gestión de respuestas tardías: se descartan y se registran.
Circuit Breaker, Timeout, Bulkhead	Clientes de Fraud, Limits, Core Integration; Legacy Core Adapter	Una dependencia lenta no debe agotar los hilos del servicio que la llama ni sobrecargar al core.	Parámetros ajustados con datos reales de carga.
Retry con backoff + Dead Letter Queue	Consumidores de eventos	Separar fallas transitorias de mensajes que nunca podrán procesarse.	Operación de la DLQ: dueño, alerta y procedimiento de reproceso.
Sidecar	Istio (mTLS) y OPA (autorización)	Seguridad homogénea sin replicar código en cada lenguaje.	Latencia adicional de pocos milisegundos y operación del mesh.
Parallel Run (dual-run)	Reconciliation Service durante migración	Comparar resultados de ambos cores antes de cambiar el SoR.	Doble procesamiento temporal en cohortes piloto.

6.2 Decisiones tecnológicas

Las decisiones se registran en formato de ADR resumido. La columna de salida indica qué tan difícil sería cambiar de proveedor, que es lo que el Banco debe sopesar.

Tabla 14. Registro resumido de decisiones de arquitectura (ADR)

Decisión	Elección	Alternativas evaluadas	Justificación	Costo de salida
Nube pública	AWS (región us-east-1, DR en us-east-2)	Azure, nube privada	Madurez de servicios administrados para Kafka y PostgreSQL, buena latencia desde Ecuador y soporte de llaves externas (BYOK).	Medio: se usan servicios con equivalente directo en Azure (tabla siguiente).
Orquestación de contenedores	Kubernetes (EKS)	ECS, serverless	Estándar portable; el mismo modelo de despliegue sirve on-premise (OpenShift) para el Legacy Core Adapter.	Bajo.
Streaming de eventos	Apache Kafka (MSK) + Schema Registry	RabbitMQ, EventBridge, Event Hubs	Retención y reprocesamiento, orden por partición, alto rendimiento y ecosistema de CDC. RabbitMQ no permite reproducir historia; EventBridge ata el diseño a AWS.	Bajo: protocolo Kafka disponible en cualquier proveedor.
API Gateway	Kong Gateway	Apigee, Azure APIM, AWS API Gateway	Funciona igual en nube y on-premise, configuración declarativa en Git (GitOps), plugins de mTLS y OIDC.	Medio: las políticas se expresan en configuración portable.
Identidad (CIAM)	Keycloak	Auth0, Entra External ID, Cognito	Soporta perfiles FAPI 2.0, PAR, DPoP, mTLS de cliente y federación; control total sobre los datos de identidad (relevante para LOPDP).	Bajo: estándares OIDC/OAuth.
Service mesh	Istio	Linkerd, sin mesh	mTLS automático con identidad SPIFFE y políticas de autorización por servicio.	Bajo.
Base de datos de servicios	PostgreSQL (Aurora)	SQL Server, DynamoDB	Relacional transaccional para estados y outbox; replicación global gestionada.	Bajo: PostgreSQL estándar.
Caché / read model	Redis (ElastiCache)	Memcached, DynamoDB	Latencia submilisegundo; el dato es reconstruible desde eventos.	Bajo.
Observabilidad	OpenTelemetry + Grafana (Tempo, Loki, Mimir)	Datadog, Dynatrace, CloudWatch	Instrumentación estándar e independiente del backend; se puede cambiar el backend sin tocar servicios.	Bajo.
Secretos y llaves	HashiCorp Vault + HSM del Banco	AWS Secrets Manager / KMS	Mismo mecanismo en nube y centro de datos; llaves maestras bajo control del Banco.	Bajo.
Lenguajes	Java 21 / Spring (dominio), Go (Fraud Screening), Node.js (BFF)	Un solo lenguaje	Java para dominio transaccional; Go donde la latencia es crítica; Node.js por cercanía con los equipos de canal.	No aplica.

Tabla 15. Equivalencias en Azure para la implementación de referencia

Componente	AWS (referencia)	Azure (equivalente)
Kubernetes administrado	EKS	AKS
Kafka administrado	MSK	Confluent Cloud en Azure / Event Hubs (API Kafka)
PostgreSQL con réplica global	Aurora PostgreSQL Global Database	Azure Database for PostgreSQL con réplicas entre regiones
CDN + WAF + DDoS	CloudFront + WAF + Shield Advanced	Front Door Premium + WAF + DDoS Protection
Conectividad dedicada	Direct Connect	ExpressRoute

Componente	AWS (referencia)	Azure (equivalente)
Conectividad privada a SaaS	PrivateLink	Private Link

7. Estrategia de APIs internas y externas

7.1 Clasificación de APIs

Las APIs se clasifican por audiencia, porque la audiencia determina el nivel de exposición, el mecanismo de seguridad y la política de cambios.

Tabla 16. Clasificación de APIs

Tipo	Consumidores	Exposición	Seguridad	Política de cambios
Experiencia (BFF)	Banca web y móvil del Banco	Internet vía gateway	OIDC + PKCE; DPoP en móvil; sesión server-side en web	Evoluciona con el canal; se coordina con la versión mínima de la app.
Externas Open Finance	Terceros autorizados	Internet vía gateway, producto separado	FAPI 2.0, mTLS, PAR, RAR, consentimiento	Versión mayor estable mínimo 12 meses; aviso de retiro con 6 meses.
Dominio (internas)	BFF, otros servicios, gateway	Red interna, mesh	mTLS SPIFFE + JWT interno de audiencia específica	Compatibilidad hacia atrás obligatoria; versión mayor nueva convive 6 meses.
Sistema (integración)	Solo Core Integration Service	Red interna / enlace dedicado	mTLS + allowlist	Cambia con el core; invisible para el resto.

7.2 Estándares de diseño y mensajería

Tabla 17. Estándares adoptados

Ámbito	Estándar y regla
Contratos de API	OpenAPI 3.1, diseño contract-first. Linting con Spectral usando un ruleset propio del Banco.
Contratos de eventos	AsyncAPI 3.0 para documentar canales; esquemas Avro registrados en Schema Registry.
Envoltura de eventos	CloudEvents 1.0 (atributos id, source, type, subject, time, dataschema) con extensiones traceparent y partitionkey.
Mensajería de pagos	ISO 20022: pain.001 (iniciación de cliente), pacs.008 (transferencia entre entidades), pacs.002 (estado), pacs.028 (consulta de estado), camt.054 (notificación de débito/crédito). ISO 8583 se mantiene solo dentro de Payment Execution para redes de tarjetas.
Datos	JSON UTF-8, camelCase; montos como string decimal con moneda ISO 4217; fechas ISO 8601 en UTC; cuentas con identificador interno opaco y número visible solo cuando el caso lo requiere.
Errores	RFC 9457 (Problem Details) con catálogo de códigos de negocio estable entre versiones.
Idempotencia	Cabecera Idempotency-Key obligatoria en todo POST con efecto monetario; retención de la clave por 24 horas.
Trazabilidad	W3C Trace Context (traceparent) en HTTP y en cabeceras de Kafka; x-fapi-interaction-id en Open Finance.
Versionado	Versión mayor en la ruta (/v1/); cambios aditivos sin cambio de versión; cabeceras Deprecation y Sunset (RFC 8594) para anunciar retiros.
Paginación y operaciones largas	Paginación por cursor; operaciones asíncronas responden 202 con Location del recurso de estado.

7.3 Ejemplos de API

Ejemplo 1 · Iniciar una orden de pago (API de dominio, BIAN Payment Order · Initiate)

```
POST /payment-order/v1/payment-orders HTTP/2
Host: payment-order.payments.svc
Authorization: Bearer eyJhbGciOiJQUzI1Nils...   (JWT interno, aud=payment-order)
Idempotency-Key: 7f9c2b4e-3d1a-4c8e-9b6f-21a5e0c4d913
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

```
Content-Type: application/json

{
  "debtorAccountId": "acc_01HZX8K2M4",
  "creditor": {
    "name": "Comercial Andina S.A.",
    "accountNumber": "2200456789",
    "bankCode": "0010",
    "accountType": "CHECKING"
  },
  "instructedAmount": { "amount": "1250.00", "currency": "USD" },
  "remittanceInformation": "Factura 001-002-000123",
  "channel": "MOBILE",
  "requestedExecution": "IMMEDIATE"
}

HTTP/2 202 Accepted
Location: /payment-order/v1/payment-orders/po_01HZX9A7QF
Content-Type: application/json

{
  "paymentOrderId": "po_01HZX9A7QF",
  "status": "FUNDS_RESERVED",
  "createdAt": "2026-10-03T15:42:10.381Z",
  "_links": { "self": "/payment-order/v1/payment-orders/po_01HZX9A7QF" }
}
```

Ejemplo 2 · Error de negocio con Problem Details (RFC 9457)

```
HTTP/2 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "https://api.banco.ec/problems/daily-limit-exceeded",
  "title": "Límite diario excedido",
  "status": 422,
  "code": "PO-LIM-002",
  "detail": "El monto supera el límite diario disponible para transferencias interbancarias.",
  "instance": "/payment-order/v1/payment-orders",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

Ejemplo 3 · Open Finance: solicitud de autorización con PAR y RAR (consentimiento)

```
POST /oauth2/par HTTP/1.1
Host: auth.banco.ec                (mTLS con certificado del tercero)
Content-Type: application/x-www-form-urlencoded

client_id=tpp-fintech-xyz
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=eyJhbGciOiJQUzI1NiIs...
&response_type=code&code_challenge_method=S256&code_challenge=Q7c...
&redirect_uri=https://fintech-xyz.ec/callback
&authorization_details=[{
   "type": "account_information",
   "permissions": ["ACCOUNTS_READ","BALANCES_READ","TRANSACTIONS_READ"],
   "accounts": ["ALL_CURRENT"],
   "transactionFromDate": "2026-04-01",
   "expirationDateTime": "2027-04-03T00:00:00Z"
 }]

HTTP/1.1 201 Created
{ "request_uri": "urn:ietf:params:oauth:request_uri:6esc_11ACC5bwc014ltc14eY22c",
  "expires_in": 60 }
```

Ejemplo 4 · Open Finance: consulta de saldos con consentimiento vigente

```
GET /open-finance/accounts/v1/accounts/acc_01HZX8K2M4/balances HTTP/1.1
Host: api.banco.ec
Authorization: DPoP eyJhbGciOiJQUzI1NiIs...   (token con consent_id y cnf ligado al certificado)
x-fapi-interaction-id: 93bac548-d2de-4546-b106-880a5018460d

HTTP/1.1 200 OK
x-fapi-interaction-id: 93bac548-d2de-4546-b106-880a5018460d
{
  "data": {
    "accountId": "acc_01HZX8K2M4",
    "availableAmount": { "amount": "3420.18", "currency": "USD" },
    "blockedAmount":  { "amount": "1250.00", "currency": "USD" },
    "asOf": "2026-10-03T15:42:11Z"
  },
  "meta": { "consentId": "cns_01HZ7Q2B9S" }
}
```

Nótese que en el ejemplo 4 la respuesta incluye asOf: el dato proviene del read model y el tercero conoce su frescura. El acceso a este endpoint lo autoriza Customer Position verificando el consentimiento en su caché local (alimentada por openfinance.consent.*); el gateway solo verifica que el token sea válido, esté ligado al certificado del tercero y tenga el scope de cuentas.

7.4 Cuotas y protección de las APIs externas

Los terceros se agrupan en planes con cuotas por segundo y por día, aplicadas por el gateway por client_id. Además, hay un límite global por cliente final para evitar que la suma de terceros conectados a una misma persona genere carga desproporcionada sobre el read model. Las APIs de pago de terceros tienen límites más bajos y requieren consentimiento de pago específico por operación o recurrente con monto máximo.

8. Arquitectura orientada a eventos

8.1 Tipos de mensajes

- **Eventos de dominio:** hechos de negocio ya ocurridos, en pasado (payment-order.completed). Son públicos para otros dominios y su esquema es un contrato.
- **Comandos asíncronos:** instrucciones dirigidas a un destinatario concreto (payment-execution.execute). Tienen un único consumidor y su topic pertenece a quien los ejecuta.
- **Eventos técnicos de integración:** cambios crudos capturados por CDC (legacy.raw.*). Son privados de la capa de integración: ningún servicio de dominio los consume directamente; solo el Legacy Event Translator.

8.2 Convenciones

- Nombre de topic: <dominio>.<entidad>.<hecho>.v<versión mayor>, por ejemplo payments.payment-order.completed.v1. El dominio coincide con la matriz de 5.2.
- Eventos delgados en datos personales: llevan identificadores (partyId, accountId) y datos de negocio, pero no nombres, cédulas, correos ni teléfonos. Quien necesite esos datos los consulta a Party Service con su propia autorización.
- Cada evento lleva aggregateVersion, un número de secuencia por agregado que permite detectar huecos o desorden en el consumidor.
- Clave de partición igual al identificador del agregado cuyo orden importa (cuenta u orden de pago).

8.3 Catálogo de eventos principales

Tabla 18. Catálogo de eventos: productores, consumidores, orden y retención

Topic	Productor	Consumidores	Clave de partición	Retención
payments.payment-order.funds-reserved.v1	Payment Order	Payment Execution (como comando), Customer Position	paymentOrderId	7 días
payments.payment-order.completed.v1	Payment Order	Notification, Customer Position, Reconciliation, Fraude, Riesgos	paymentOrderId	30 días
payments.payment-order.rejected.v1	Payment Order	Notification, Customer Position, Reconciliation	paymentOrderId	30 días
payments.payment-execution.completed.v1 / failed.v1	Payment Execution	Payment Order, Reconciliation	paymentOrderId	30 días
accounts.transaction.posted.v1	Legacy / Digital Event Translator	Customer Position, Fraude (stream), Riesgos, Notification	accountId	30 días (compactado aparte)
accounts.balance.changed.v1	Legacy / Digital Event Translator	Customer Position	accountId	Compactado
party.party.updated.v1	Party Service	Core Integration (sincroniza cores), Notification	partyId	Compactado
openfinance.consent.granted.v1 / revoked.v1	Consent Service	Customer Position, Payment Order, Servidor de Identidad (revoca tokens)	consentId	Compactado
fraud.alert.raised.v1	Fraud Screening	Notification, Payment Order (órdenes en revisión)	partyId	30 días
risk.limit.changed.v1	Limits Service	Payment Order (invalida caché)	partyId	Compactado
core.routing.changed.v1	Core Integration	Customer Position, Reconciliation	accountId	Compactado
migration.cohort.migrated.v1	Core Integration	Reconciliation, Notification, Customer Position	cohortId	1 año

8.4 Payloads de referencia

Evento payments.payment-order.completed.v1 (envoltura CloudEvents, cuerpo según esquema Avro)


```
{
 "specversion": "1.0",
 "id": "evt_01HZX9F3K8T2",
 "source": "urn:banco:payments:payment-order-service",
 "type": "ec.banco.payments.payment-order.completed.v1",
 "subject": "po_01HZX9A7QF",
 "time": "2026-10-03T15:42:13.902Z",
 "datacontenttype": "application/avro",
 "dataschema": "registry://payments.payment-order.completed/versions/3",
 "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-a1b2c3d4e5f60718-01",
 "partitionkey": "po_01HZX9A7QF",
 "data": {
  "paymentOrderId": "po_01HZX9A7QF",
  "aggregateVersion": 6,
  "debtorAccountId": "acc_01HZX8K2M4",
  "debtorPartyId": "pty_01HX2M0QZ7",
  "amount": { "amount": "1250.00", "currency": "USD" },
  "rail": "SPI",
  "railReference": "SPI20261003000184532",
  "servicingCore": "LEGACY",
  "completedAt": "2026-10-03T15:42:13.880Z"
 }
}
```


Evento accounts.transaction.posted.v1 originado por CDC del core tradicional


```
{
 "specversion": "1.0",
 "id": "evt_01HZX9F4A1M0",
 "source": "urn:banco:integration:legacy-event-translator",
 "type": "ec.banco.accounts.transaction.posted.v1",
 "subject": "acc_01HZX8K2M4",
 "time": "2026-10-03T15:42:13.951Z",
 "partitionkey": "acc_01HZX8K2M4",
 "data": {
  "accountId": "acc_01HZX8K2M4",
  "aggregateVersion": 18342,
  "transactionId": "txn_LEG_2026100300918231",
  "externalReference": "po_01HZX9A7QF",
  "direction": "DEBIT",
  "amount": { "amount": "1250.00", "currency": "USD" },
  "balanceAfter": { "amount": "2170.18", "currency": "USD" },
  "bookingDate": "2026-10-03",
  "servicingCore": "LEGACY",
  "sourceCommitTs": "2026-10-03T15:42:13.612Z"
 }
}
```


El campo externalReference lleva el identificador de la orden de pago hasta el core y de vuelta. Es lo que permite al Reconciliation Service unir la orden, la ejecución en el rail y el movimiento contable sin heurísticas.

8.5 Versionado de eventos

- Modo de compatibilidad BACKWARD_TRANSITIVE en Schema Registry: un consumidor con el esquema nuevo puede leer todos los eventos anteriores. El pipeline rechaza el registro de un esquema incompatible.
- Cambios permitidos sin versión mayor: agregar campos opcionales con valor por defecto, agregar valores a un enum solo si los consumidores declaran tolerancia (regla del catálogo).
- Cambios incompatibles (renombrar, eliminar, cambiar tipo o significado) generan un topic nuevo .v2. El productor publica en ambos durante la ventana de migración (máximo 6 meses) y el catálogo muestra qué consumidores siguen en v1.
- El significado de un campo no se cambia nunca sin versión mayor, aunque el tipo se mantenga. Este es el error más difícil de detectar por herramientas, por eso se exige revisión del dueño del dominio.

8.6 Reintentos y dead-letter queues

La estrategia de reintento depende de si el consumidor necesita orden estricto. Mezclar ambas estrategias es una fuente habitual de errores, así que cada consumidor declara su tipo en el catálogo.

Tabla 19. Estrategia de reintento según tipo de consumidor

Aspecto	Consumidor sin requisito de orden (p. ej. Notification)	Consumidor con orden por clave (p. ej. Customer Position)
Falla transitoria	Reintento no bloqueante: el mensaje pasa a topics de reintento con espera creciente (1 min, 10 min, 1 h) y la partición sigue avanzando.	Reintento bloqueante en memoria con backoff exponencial y jitter (5 intentos, hasta 30 s), para no adelantar eventos posteriores de la misma cuenta.
Falla persistente	Tras el último reintento, a <topic>.dlq.	El evento va a la DLQ y la clave queda "estacionada": los siguientes eventos de esa cuenta se desvían a un topic de espera hasta resolver el primero. Las demás cuentas siguen fluyendo.
Error no recuperable (esquema inválido, regla de negocio)	Directo a DLQ, sin reintentos.	Directo a DLQ con estacionamiento de clave.
Contenido de la DLQ	Mensaje original intacto + cabeceras: error, stack resumido, número de intentos, servicio consumidor, traceparent.	Igual, más el offset de origen para reprocesar en orden.
Operación	Alerta al equipo dueño. Herramienta de reproceso con aprobación y registro en auditoría.	Alerta de severidad alta: una cuenta estacionada implica saldo desactualizado en canales.

8.7 Ordenamiento e idempotencia

- **Orden:** Kafka garantiza orden solo dentro de una partición. Por eso la clave de partición es la cuenta para movimientos y la orden de pago para eventos de pago. Los productores usan enable.idempotence=true y max.in.flight.requests.per.connection≤5, lo que mantiene el orden ante reintentos del productor.
- **Detección de desorden:** el consumidor compara aggregateVersion con la última versión procesada para esa clave. Si llega una versión menor o igual, es un duplicado y se descarta. Si hay un salto, se espera brevemente y luego se marca la clave como pendiente de reconstrucción desde el SoR.
- **Idempotencia en el consumidor:** tabla inbox con el id del evento como clave única, escrita en la misma transacción que el efecto de negocio. Un evento repetido falla la inserción y se ignora.
- **Idempotencia hacia los cores:** toda operación monetaria lleva una clave de operación (paymentOrderId + paso de la saga). Ambos adaptadores la envían al core como referencia externa y, ante una respuesta ambigua, consultan por esa referencia antes de reintentar.
- **Idempotencia hacia el cliente:** cabecera Idempotency-Key en la API, con almacenamiento de la respuesta original.

La combinación de outbox, productor idempotente, inbox y claves de operación da un efecto de procesamiento exactamente una vez a nivel de negocio, aunque el transporte sea al menos una vez.

8.8 Configuración de referencia de Kafka

Tabla 20. Parámetros de referencia del Event Backbone

Parámetro	Valor	Motivo
replication.factor	3	Una réplica por zona de disponibilidad.
min.insync.replicas	2	Tolera la pérdida de una zona sin perder escrituras confirmadas.
acks (productores)	all	Confirmación solo cuando las réplicas en sincronía tienen el dato.

Parámetro	Valor	Motivo
Particiones en topics de cuentas	48	Dimensionado para el pico de 600 TPS con margen y paralelismo de consumidores.
Formato	Avro + Schema Registry	Contratos verificables y mensajes compactos.
Cifrado	TLS en tránsito, cifrado en reposo con llave del Banco	Requisito de seguridad y LOPDP.
Autorización	ACL por cuenta de servicio y topic (prefijos)	Un servicio solo escribe en los topics de su dominio.

9. Estrategia de integración multicore

9.1 El problema a resolver

Durante varios años coexistirán dos cores con modelos de datos, protocolos, capacidades y horarios de cierre distintos. Un mismo cliente puede tener una cuenta corriente en el core tradicional y una cuenta de ahorro digital en el nuevo. Los canales, los terceros y los servicios de dominio no deben saberlo, y el Banco debe poder mover clientes de un core a otro sin que esos consumidores cambien.

9.2 Elementos de la estrategia y dónde están en el diseño

Tabla 21. Elementos de la estrategia multicore y su ubicación

Elemento	Descripción	Ubicación en el diseño
Fachada única de cuentas	Todas las operaciones sobre cuentas pasan por una sola API con semántica BIAN, sin importar el core.	Core Account API (fig. 4).
Ruteo por datos	El core destino se resuelve por cuenta, producto y cohorte, con reglas versionadas y fecha efectiva.	Core Routing Policy + Routing Registry.
ACL por core	Cada core tiene su traductor al modelo canónico. Los errores del core se mapean a un catálogo canónico.	Digital Core Adapter; Legacy Core Adapter on-premise.
Puertos segregados	Las capacidades se declaran por puerto; el ruteo sabe qué core soporta qué.	AccountQueryPort, PostingPort, HoldPort, PartyPort.
Identificadores globales	Las cuentas se identifican con un id interno opaco (acc_...) que no cambia al migrar. El número de cuenta visible se conserva cuando el producto lo permite.	Routing Registry guarda id global ↔ (core, id nativo).
Eventos unificados	Los movimientos de ambos cores llegan como el mismo evento canónico.	Legacy y Digital Event Translator.
Vista consolidada	El cliente ve todos sus productos juntos.	Customer Position Service.
Operaciones entre cores	Transferencias con origen y destino en cores distintos.	Cross-Core Transfer Coordinator.
Conciliación continua	Saldos, movimientos y cuentas puente se concilian intradía y al cierre.	Reconciliation Service.

9.3 Reglas de ruteo

Las reglas se evalúan de la más específica a la más general. El ejemplo muestra cómo conviven reglas de producto, de cohorte y de cuenta individual durante la migración:

Tabla 22. Ejemplo de reglas del Routing Registry

Prioridad	Nivel	Criterio	Core	Vigencia
1	Cuenta	acc_01HZX8K2M4 (excepción por incidente)	LEGACY	Hasta resolución, con aprobación dual
2	Cohorte	Cohorte C-03: colaboradores del Banco	DIGITAL	Desde 2027-03-01 00:00
3	Producto	Ahorro digital (AHD)	DIGITAL	Desde lanzamiento
4	Producto	Cuenta corriente tradicional (CTE)	LEGACY	Hasta migración de cartera
5	Por defecto	Cualquier otro	LEGACY	Permanente durante la transición

El registro se cachea en memoria del Core Integration Service y se invalida con core.routing.changed.v1. Un cambio de regla no aplica a operaciones en curso: cada saga fija el core al inicio para evitar que una operación quede repartida entre dos cores.

9.4 Transferencia entre cuentas de cores distintos

Cuando el origen está en el core tradicional y el destino en el digital (o al revés) no existe una transacción única posible. El Cross-Core Transfer Coordinator usa una cuenta puente en cada core, que es la práctica contable habitual en bancos con más de un core:

1. Débito a la cuenta del cliente y crédito a la cuenta puente en el core origen (una sola transacción local, con la clave de operación).
2. Débito a la cuenta puente y crédito a la cuenta destino en el core destino (otra transacción local, con la misma clave y sufijo de paso).
3. Si el paso 2 falla de forma definitiva, se reversa el paso 1 dentro del mismo día contable.
4. Las cuentas puente deben quedar en cero al final de cada ventana de conciliación; cualquier saldo residual genera una alerta al área de operaciones con el detalle de las órdenes involucradas.

9.5 Diferencias de capacidad y horarios

Los cores tienen ventanas de cierre distintas. Durante el cierre del core tradicional el Legacy Core Adapter responde con un código canónico de "core en cierre" y la saga mantiene la orden en espera (para pagos programados) o la rechaza con código reintentable (para pagos inmediatos), según la política de producto. El canal informa al cliente con el mismo mensaje, independiente del core.

10. Gestión de identidad y acceso

10.1 Dominios de identidad

Tabla 23. Tipos de identidad y mecanismos

Identidad	Proveedor	Protocolo	Autenticación	Credencial resultante
Cliente	Keycloak (realm clientes)	OIDC Authorization Code + PKCE	Usuario + factor de posesión (dispositivo vinculado) + biometría local; step-up por riesgo y monto	Access token 5 min ligado por DPoP; refresh token rotativo ligado al dispositivo
Tercero Open Finance	Keycloak (realm open-finance)	OAuth 2.0 FAPI 2.0: PAR, RAR, private_key_jwt	Certificado de cliente (mTLS) registrado en el directorio de participantes	Access token ligado al certificado (RFC 8705) con consent_id
Colaborador	IdP corporativo federado en Keycloak	OIDC / SAML	SSO + MFA resistente a phishing (FIDO2)	Token con roles y atributos (agencia, nivel de aprobación)
Servicio (workload)	Istio CA / SPIFFE	mTLS	Certificado X.509 de corta duración (24 h) emitido automáticamente	Identidad SPIFFE spiffe://banco.ec/ns/<ns>/sa/<servicio>
Sistema legado	Vault	Usuario técnico MQ con TLS	Credencial dinámica rotada	Credencial por tipo de operación

10.2 Autenticación de clientes

- **Web:** el BFF Web actúa como cliente confidencial. Los tokens nunca llegan al navegador; el navegador solo tiene una cookie de sesión HttpOnly, Secure y SameSite=Strict. Esto elimina el robo de tokens por XSS.
- **Móvil:** en el enrolamiento la app genera un par de llaves en el almacén seguro del dispositivo (Secure Enclave / StrongBox) y registra la llave pública. Cada token queda ligado a esa llave mediante DPoP, por lo que un token robado no sirve desde otro dispositivo.
- **Biometría:** se usa solo para desbloquear la llave del dispositivo. Ninguna plantilla biométrica sale del teléfono, lo cual es una decisión de protección de datos y no solo técnica (sección 11.3).
- **Step-up y firma de transacción:** para beneficiarios nuevos o montos sobre el umbral, el servidor exige un acr superior. La app firma con la llave del dispositivo un resumen que incluye monto y beneficiario (dynamic linking), y Payment Order verifica la firma. Así la autorización queda atada a la operación concreta.
- **Señales de riesgo:** el Servidor de Identidad consulta al sistema de fraude en el login (dispositivo, geolocalización, velocidad) y puede exigir un factor adicional.

10.3 Terceros Open Finance

Un tercero se registra con un software statement emitido por el directorio de participantes (registro dinámico de clientes). Cada solicitud de autorización pasa por PAR, lo que evita manipulación de parámetros en el navegador; el detalle del consentimiento viaja en authorization_details (RAR), y el Consent Service lo persiste. El cliente autentica y aprueba en el Banco, nunca en el tercero. Al revocar un consentimiento, el evento openfinance.consent.revoked.v1 revoca los tokens asociados y actualiza las cachés de los servicios en segundos.

10.4 Colaboradores

Los colaboradores usan la identidad corporativa federada. Los permisos combinan roles (RBAC) con atributos (ABAC) como agencia, monto máximo de aprobación y segregación de funciones (quien registra no aprueba). El acceso administrativo a producción pasa por una solución de acceso privilegiado con elevación temporal, aprobación y grabación de sesión; existe un procedimiento de emergencia (break-glass) con cuentas selladas y auditadas.

10.5 Servicio a servicio

- Todo el tráfico este-oeste usa mTLS del mesh con identidad SPIFFE. Las políticas de autorización del mesh son deny-by-default y reproducen exactamente las flechas del diagrama de contenedores: si una relación no está en el diagrama, no está permitida en la red.
- El token externo del cliente no se propaga hacia adentro. El BFF (o el gateway, para Open Finance) hace un intercambio de token (RFC 8693) y obtiene un JWT interno con audiencia del servicio destino, el partyId del cliente, el acr y, si aplica, el consent_id. Cada servicio valida el JWT localmente con las llaves publicadas, sin introspección remota.
- Las credenciales de base de datos y de Kafka son dinámicas, emitidas por Vault para la cuenta de servicio dueña. Esto hace técnicamente imposible que un servicio lea la base de otro.

10.6 Autorización en capas

Tabla 24. Control de acceso por capa de la arquitectura

Capa	Qué decide	Mecanismo	Ejemplo
API Gateway	Si el token es válido, para esta API, ligado al emisor correcto y dentro de cuota	Plugins OIDC, mTLS y rate limiting	Token sin scope payments → 403 en el borde.
Service mesh	Si el servicio que llama puede invocar al servicio llamado	AuthorizationPolicy de Istio con identidades SPIFFE	Notification no puede invocar a Payment Order.
Servicio (PEP + OPA)	Si el sujeto puede operar sobre este recurso concreto	Políticas Rego versionadas; datos de contexto locales	El partyId del token es titular de la cuenta origen; el consentimiento cubre la cuenta.
Dominio	Si la operación es válida para el negocio	Invariantes del agregado	No se puede cancelar una orden ya liquidada.
Datos	Qué servicio accede a qué base y con qué privilegios	Credenciales dinámicas de Vault; cifrado por llave de dominio	Party DB solo accesible por la cuenta de servicio de Party.
Core tradicional	Que solo el adaptador opere	Canal MQ con TLS y reglas de autenticación por canal; allowlist de red	Una conexión desde otra IP es rechazada.

11. Seguridad, cumplimiento normativo y protección de datos personales

11.1 Marco normativo considerado

- **Ley Orgánica de Protección de Datos Personales (LOPDP, 2021) y su Reglamento.** Rige el tratamiento de datos de clientes, colaboradores y terceros.
- **Normativa de la Superintendencia de Bancos** sobre gestión del riesgo operativo, seguridad de la información, seguridad de canales electrónicos y continuidad del negocio.
- **Prevención de lavado de activos:** obligaciones de reporte a la UAFE, que se apoyan en el sistema de prevención de fraudes y en la trazabilidad de las transacciones.
- **PCI DSS v4.0** para cualquier componente que procese datos de tarjetas.
- **ISO/IEC 27001:2022** como marco de gestión de seguridad y **estándares Open Finance** (FAPI 2.0) mientras se publica la norma local.

11.2 Clasificación de datos

Tabla 25. Niveles de clasificación y controles mínimos

Nivel	Ejemplos	Controles mínimos
Restringido	Datos biométricos, PAN de tarjeta, credenciales, llaves	No se almacenan fuera de su sistema dueño (biometría solo en el dispositivo; PAN tokenizado). Cifrado con llave dedicada. Acceso registrado.
Confidencial	Identificación, contacto, saldos, movimientos, consentimientos	Cifrado en reposo con llave por dominio; excluidos de logs y de eventos (solo identificadores); acceso por necesidad.
Interno	Identificadores opacos, estados de órdenes, métricas	Cifrado en reposo estándar; acceso por rol.
Público	Tarifario, ubicación de agencias	Integridad y disponibilidad.

11.3 Elementos de la LOPDP y su presencia en la solución

La LOPDP no se cumple con un componente; se cumple con decisiones repartidas en la arquitectura. La tabla recorre los elementos de la ley y señala dónde está cada uno en el diseño.

Tabla 26. Elementos de la LOPDP y su materialización en la arquitectura

Elemento de la LOPDP	Qué exige	Cómo y dónde está en la solución
Principios: finalidad, pertinencia y minimización	Tratar solo los datos necesarios para una finalidad determinada.	Eventos delgados sin datos personales; el BFF solo pide los campos que la pantalla muestra; las APIs Open Finance devuelven exactamente los permisos consentidos (RAR).
Licitud y consentimiento	Base legítima para cada tratamiento; consentimiento libre, específico, informado e inequívoco cuando es la base.	Consent Service guarda alcance, finalidad, vigencia y evidencia de aceptación. Los tratamientos por relación contractual u obligación legal se documentan en el registro de actividades.
Derechos de acceso, rectificación, eliminación, oposición, portabilidad y suspensión	Atenderlos dentro de los plazos legales.	Party Service es el punto único para rectificar (el cambio se propaga por evento a ambos cores). Un flujo de derechos usa el catálogo de datos para ubicar todas las copias. La portabilidad se apoya en las mismas APIs de datos de Open Finance.
Decisiones automatizadas	Derecho a no ser objeto de decisiones basadas únicamente en tratamiento automatizado con efectos significativos, y a obtener explicación.	Las decisiones de fraude en estado REVIEW pasan por un analista. Las declinaciones automáticas registran los motivos (reglas y variables principales del score) para explicación y revisión humana a pedido del cliente.
Datos sensibles	Protección reforzada (por ejemplo, datos biométricos).	La biometría se procesa solo en el dispositivo del cliente; el Banco recibe una firma criptográfica, no la huella ni el rostro.

Elemento de la LOPDP	Qué exige	Cómo y dónde está en la solución
Seguridad del tratamiento	Medidas técnicas y organizativas apropiadas al riesgo.	Tabla de controles 11.4: cifrado, mTLS, autorización en capas, tokenización, gestión de llaves, monitoreo.
Notificación de vulneraciones	Notificar a la Autoridad y, cuando corresponda, a los titulares dentro del término legal.	SIEM con casos de uso de exfiltración; la bitácora de auditoría permite determinar qué titulares fueron afectados; procedimiento de respuesta con el Delegado de Protección de Datos.
Evaluación de impacto (EIPD)	Evaluar tratamientos de alto riesgo antes de implementarlos.	EIPD obligatoria para Open Finance, scoring de fraude, uso de nube y migración de datos entre cores; sus controles se trazan a esta arquitectura.
Encargados del tratamiento	Contratos que garanticen el cumplimiento por terceros que tratan datos por cuenta del Banco.	El proveedor del core digital, el proveedor de nube y los proveedores de notificación son encargados: contratos, auditorías y cifrado con llaves del Banco.
Transferencia internacional	Garantías adecuadas al transferir datos fuera del país.	La región de nube está fuera de Ecuador: se documenta la transferencia, se aplican cláusulas contractuales y los datos se cifran con llaves custodiadas en el HSM del Banco (BYOK), de modo que el proveedor no puede leerlos.
Delegado de Protección de Datos y responsabilidad proactiva	Designar un delegado y poder demostrar cumplimiento.	Catálogo de datos con linaje, registro de accesos a datos confidenciales en el Audit Trail y reportes periódicos al delegado.
Conservación y eliminación	Conservar solo el tiempo necesario.	Políticas de retención por clase de dato. Como la normativa bancaria obliga a conservar registros transaccionales, la eliminación aplica tras ese plazo; para datos en logs inmutables se usa crypto-shredding (destrucción de la llave del titular).

11.4 Tabla de controles de seguridad por ubicación

Tabla 27. Controles de seguridad ubicados en la arquitectura

Ubicación	Control	Implementación	Amenaza mitigada	Referencia
CDN/WAF	Protección perimetral	Reglas OWASP, bot management, DDoS L3-L7, TLS 1.3	DDoS, inyección, credential stuffing	SB canales electrónicos
API Gateway	Autenticación de cliente de API	Validación JWT, DPoP, mTLS de terceros, cuotas	Suplantación, abuso de API	FAPI 2.0
Servidor de Identidad	Autenticación fuerte	MFA, device binding, step-up, firma de transacción	Toma de cuenta, fraude por ingeniería social	SB, LOPDP
BFF Web	Gestión segura de sesión	Tokens server-side, cookie HttpOnly/SameSite, CSRF	Robo de tokens por XSS	OWASP ASVS
Service mesh	Confianza cero este-oeste	mTLS SPIFFE, AuthorizationPolicy deny-by-default	Movimiento lateral	ISO 27001 A.8.20
Servicios de dominio	Autorización fina	OPA/ABAC: titularidad, consentimiento, montos	Acceso a datos de otro cliente (BOLA)	OWASP API Top 10
Payment Order	Integridad de la operación	Idempotencia, firma de transacción, verificación de dynamic linking	Duplicidad, manipulación de monto	SB
Bases de datos	Cifrado y aislamiento	Cifrado en reposo con llave por dominio, credenciales dinámicas	Fuga de datos, acceso cruzado	LOPDP, PCI DSS
Event Backbone	Confidencialidad de eventos	TLS, ACL por topic, eventos sin PII	Exposición de datos en tránsito	LOPDP
Payment Execution	Reducción de alcance PCI	Tokenización del PAN; datos de tarjeta solo en la plataforma de pagos	Robo de datos de tarjeta	PCI DSS v4.0
Legacy Core Adapter	Protección del core	Bulkhead, rate limit, canal MQ con TLS	Denegación de servicio al core	SB riesgo operativo
Enlaces dedicados	Cifrado nube–centro de datos	MACsec / IPsec sobre Direct Connect; mTLS de aplicación	Intercepción	ISO 27001
Vault + HSM	Gestión de llaves y secretos	Llaves maestras en HSM, rotación automática, secretos dinámicos	Secretos expuestos en código	PCI DSS, ISO 27001

Ubicación	Control	Implementación	Amenaza mitigada	Referencia
Collector OpenTelemetry	Datos personales fuera de la telemetría	Procesador de enmascaramiento de cédula, correo, teléfono, número de cuenta	Fuga por logs	LOPDP
Audit Trail	No repudio	Registro append-only, encadenado por hash, almacenamiento WORM	Alteración de evidencia	SB, LOPDP
Pipeline CI/CD	Cadena de suministro	SAST, SCA, SBOM, firma de imágenes, admisión solo de imágenes firmadas	Código o dependencias maliciosas	ISO 27001 A.8.28
SIEM / SOC	Detección y respuesta	Correlación de eventos de IdP, gateway, mesh y auditoría	Ataques en curso, exfiltración	SB, LOPDP

11.5 Protección de datos personales en eventos, logs y entornos no productivos

- Los eventos solo transportan identificadores opacos. Si un consumidor necesita un dato personal, lo consulta a Party Service, que registra el acceso.
- Los logs pasan por el collector, que enmascara patrones de datos personales antes de enviarlos al backend. Los equipos no tienen acceso a logs sin enmascarar.
- Los entornos de desarrollo y pruebas usan datos sintéticos o enmascarados irreversiblemente; nunca copias de producción.
- Para datos que deben borrarse dentro de almacenamientos inmutables (por ejemplo, auditoría ya expirada) se cifra cada registro con una llave asociada al titular y se destruye la llave al vencer la retención.

12. Alta disponibilidad y recuperación ante desastres

12.1 Clasificación por criticidad

No todos los componentes necesitan el mismo nivel de protección. Clasificar por niveles permite invertir donde el impacto lo justifica.

Tabla 28. Niveles de criticidad, objetivos y componentes

Nivel	Componentes	RTO	RPO	Estrategia
N0	Servidor de Identidad, API Gateway, BFF, Payment Order, Core Integration, Legacy Core Adapter, Event Backbone	15 min en región / 30 min entre regiones	0 en región / ≤ 1 min entre regiones	Multi-AZ activo; warm standby en segunda región.
N1	Customer Position, Party, Consent, Payment Execution, Fraud Screening, Limits	30 min	≤ 1 min	Multi-AZ; Customer Position se reconstruye desde eventos.
N2	Notification, Reconciliation, observabilidad	4 h	≤ 15 min	Multi-AZ; recuperación desde respaldos o reproceso.

12.2 Topología de despliegue

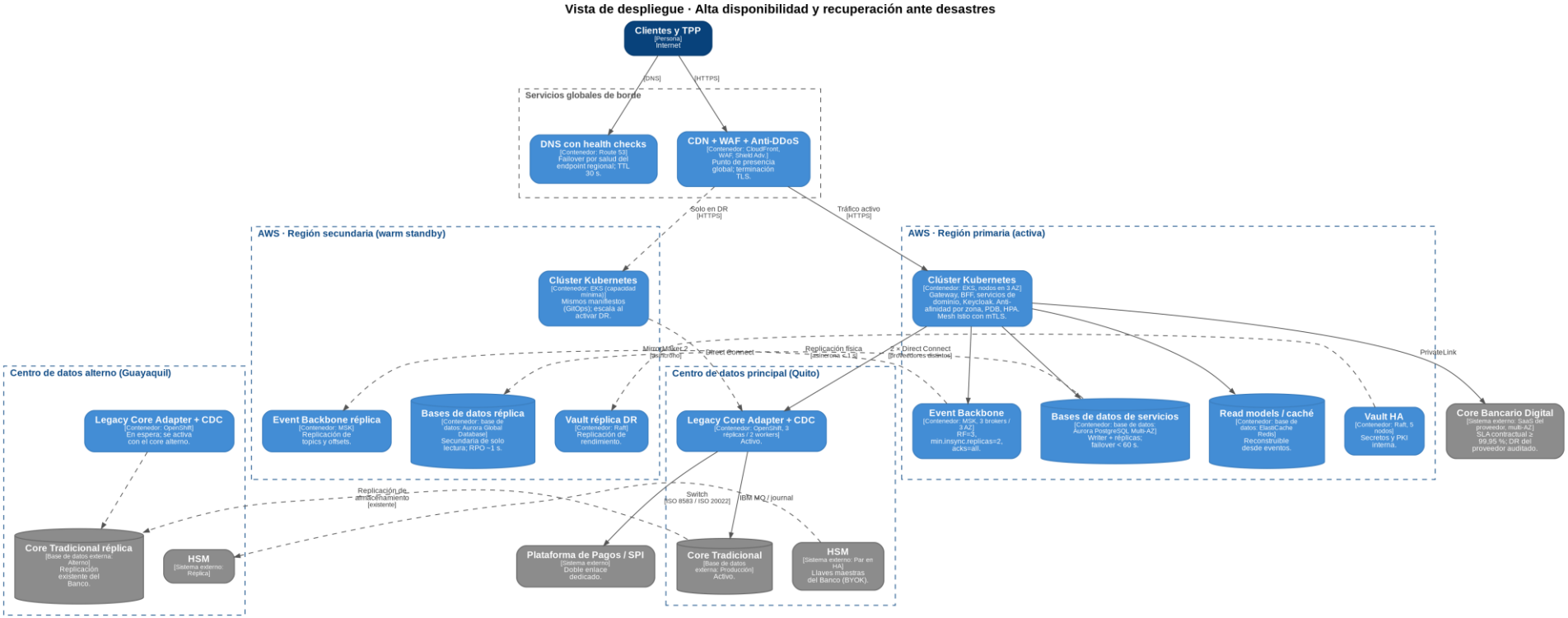


Figura 6. Vista de despliegue para alta disponibilidad y recuperación ante desastres

La topología combina tres niveles de protección. Dentro de la región primaria, todo se despliega en tres zonas de disponibilidad. Entre regiones, la secundaria se mantiene en warm standby con datos replicados. En el lado on-premise se reutiliza la arquitectura de centro de datos principal y alterno que el Banco ya tiene para el core tradicional, y el Legacy Core Adapter se despliega en ambos.

12.3 Alta disponibilidad dentro de la región

Tabla 29. Mecanismos de alta disponibilidad por componente

Componente	Mecanismo	Comportamiento ante pérdida de una zona
Servicios en Kubernetes	Mínimo 3 réplicas con anti-afinidad por zona, PodDisruptionBudget, HPA por CPU y por lag de consumo	Las réplicas restantes absorben tráfico; HPA repone capacidad.
API Gateway	Modo DB-less con configuración declarativa: no tiene estado que replicar	Sin impacto.
Servidor de Identidad	Clúster con caché distribuida y base PostgreSQL Multi-AZ	Las sesiones activas se conservan.
Event Backbone	Brokers en 3 zonas, RF=3, min.insync=2	Sin pérdida de mensajes confirmados; los líderes se reubican.
PostgreSQL	Instancia escritora con réplicas en otras zonas	Failover automático en menos de 60 s; los servicios reintentan.
Redis (read models)	Modo clúster con réplica por shard	Promoción de réplica; si se perdiera, se reconstruye desde eventos.
Enlaces nube–centro de datos	Dos Direct Connect con proveedores distintos + VPN IPsec de respaldo	Conmutación automática por BGP.

12.4 Recuperación entre regiones

Se eligió **warm standby** y no activo-activo entre regiones. La razón principal es que los sistemas de registro (ambos cores) son de un solo escritor, y las órdenes de pago necesitan un único lugar donde su estado sea autoritativo. Un esquema activo-activo obligaría a resolver conflictos de escritura sobre órdenes de pago, lo que agrega riesgo sin beneficio real, porque la latencia ya se cumple desde una sola región. El costo del standby se mantiene bajo porque la región secundaria corre con capacidad mínima y escala solo al activarse.

El procedimiento de activación está automatizado y lo declara el comité de crisis, no un umbral automático, para evitar conmutaciones por falsos positivos:

1. Se confirma la pérdida de la región primaria (health checks de DNS, sondas sintéticas y estado del proveedor).
2. Se promueve la base de datos secundaria a escritora y se activa el clúster Kafka secundario con offsets traducidos por MirrorMaker 2.
3. Se escala el clúster Kubernetes secundario; los manifiestos ya están desplegados por GitOps.
4. El DNS conmuta el tráfico. Los clientes con sesión activa deben volver a autenticarse si la réplica de sesiones no alcanzó a completarse.
5. Payment Order ejecuta un barrido de sagas en estado no final y consulta su estado real al rail y al core antes de continuar. La idempotencia por clave de operación evita duplicar efectos.
6. Reconciliation Service ejecuta una conciliación extraordinaria del intervalo afectado.

El retorno a la región primaria (failback) se hace en una ventana planificada, en sentido inverso y con el mismo barrido de sagas.

12.5 Escenarios de falla y comportamiento esperado

Tabla 30. Escenarios de falla, comportamiento del sistema e impacto en el cliente

Escenario	Comportamiento del sistema	Impacto en el cliente	Recuperación
Caída de una zona de disponibilidad	Redistribución automática de pods, líderes de Kafka y failover de bases.	Ninguno perceptible; algunas solicitudes reintentadas.	Automática.
Caída de la región primaria	Activación de warm standby (12.4).	Interrupción de hasta 30 minutos.	Procedimiento DR.
Core tradicional no disponible	Circuit breaker abierto en Legacy Core Connector. Consultas desde read model con hora del dato. Operaciones monetarias de cuentas del core tradicional rechazadas con código reintentable; pagos programados en espera.	Ve saldos con marca de hora; no puede transferir desde esas cuentas. Las cuentas del core digital siguen operando.	Al cerrar el circuito, la saga reanuda programados.
Core digital no disponible	Simétrico al anterior para cuentas del core digital.	Simétrico.	Según SLA del proveedor.
Sistema de fraude lento o caído	Al vencer 150 ms, Fraud Screening aplica reglas de contingencia locales con límites reducidos y marca la orden para revisión posterior.	Transferencias pequeñas a beneficiarios conocidos siguen funcionando; montos altos se rechazan temporalmente.	Reevaluación posterior de lo aprobado en contingencia.
Plataforma de pagos o SPI no disponible	Órdenes en SUBMITTED esperan dentro de la ventana de saga; luego IN_INQUIRY y consulta antes de liberar fondos.	Orden pendiente con fondos retenidos; notificación si se rechaza.	Reanudación automática.
Event Backbone degradado	Los servicios siguen escribiendo en su outbox; el relay publica cuando Kafka se recupera.	Notificaciones y saldos del read model con retraso.	Sin pérdida de eventos.
Servidor de Identidad no disponible	Tokens vigentes siguen siendo válidos (validación local con JWKS en caché).	No pueden iniciarse sesiones nuevas.	Multi-AZ; DR con la región.
Corte de un enlace dedicado	BGP conmuta al segundo enlace; si ambos fallan, VPN de respaldo.	Mayor latencia en operaciones del core tradicional.	Automática.

12.6 Pruebas de resiliencia

- Pruebas de caos mensuales en preproducción (pérdida de pods, de zona y de dependencias) y trimestrales en producción en horario de bajo tráfico, empezando por componentes N2.
- Simulacro completo de DR entre regiones dos veces al año, con medición real de RTO y RPO y acta para el plan de continuidad que exige la Superintendencia de Bancos.
- Pruebas de carga antes de cada fecha pico (quincena, fin de mes, décimo tercer sueldo) con 1,5 veces el pico histórico.

13. Gobierno de APIs y microservicios

El gobierno se diseña para que la mayoría de las reglas se verifiquen automáticamente en el pipeline. Un comité que revisa todo manualmente se convierte en cuello de botella; uno que no revisa nada deja que el diseño se degrade. El punto medio es automatizar lo verificable y reservar la revisión humana para decisiones de diseño.

13.1 Modelo organizativo

Tabla 31. Roles del modelo de gobierno

Rol	Responsabilidad
Autoridad de diseño (arquitectura)	Define estándares, aprueba nuevos servicios y cambios incompatibles, mantiene la matriz de dominios y el registro de ADR. Se reúne semanalmente; las decisiones quedan escritas.
Dueño de dominio	Responsable funcional y técnico de los servicios, APIs y eventos de su dominio. Aprueba cambios de contrato y responde por sus SLO.
Equipo de plataforma	Opera gateway, mesh, Kafka, observabilidad y pipelines; provee plantillas de servicio que ya cumplen los estándares.
Product manager de APIs externas	Gestiona el portafolio Open Finance: roadmap, comunicación con terceros, versiones y retiro.
Seguridad de la información	Define controles, revisa diseños de alto riesgo y gestiona pruebas de penetración.
Delegado de Protección de Datos	Valida EIPD, finalidades y retenciones de nuevos tratamientos.

Tabla 32. Matriz RACI de actividades de gobierno

Actividad	Aut. diseño	Dueño dominio	Plataforma	PM APIs	Seguridad	DPD
Crear un microservicio nuevo	A	R	C	I	C	I
Cambio compatible de API o evento	I	A/R	I	I	—	—
Cambio incompatible de API o evento	A	R	C	C	I	—
Publicar API externa	C	R	C	A	C	C
Nuevo tratamiento de datos personales	C	R	—	I	C	A
Retiro de versión de API	C	R	I	A	—	—

13.2 Ciclo de vida de una API o evento

1. **Diseño:** el contrato (OpenAPI o AsyncAPI + Avro) se escribe antes que el código, en el repositorio de contratos, con el service domain BIAN y el dueño como metadatos obligatorios.
2. **Revisión:** el linting automático valida nomenclatura, seguridad declarada, errores RFC 9457 y paginación. La revisión humana se enfoca en el modelo de dominio.
3. **Implementación y pruebas:** pruebas de contrato del consumidor (Pact) y validación de compatibilidad de esquemas bloquean el merge si fallan.
4. **Publicación:** el contrato aparece en el catálogo interno (Backstage) y, si es externo, en el portal de desarrolladores con sandbox y datos sintéticos.
5. **Operación:** cada API tiene SLO publicado y tablero; el dueño recibe las alertas.
6. **Deprecación y retiro:** se anuncia con cabeceras Deprecation y Sunset, se monitorea quién sigue consumiendo la versión anterior y se retira solo cuando el consumo llega a cero o vence el plazo comunicado.

13.3 Reglas verificadas automáticamente (fitness functions)

Tabla 33. Reglas de arquitectura verificadas en el pipeline o en operación

Regla	Cómo se verifica	Dónde
Nomenclatura de rutas, campos y topics	Ruleset Spectral del Banco; validación de nombres de topic en Terraform	Pull request
Compatibilidad de esquemas de eventos	Schema Registry en modo BACKWARD_TRANSITIVE	Pipeline de contratos
Sin base de datos compartida	Cada credencial de BD se emite para una única cuenta de servicio; política de Vault revisada en IaC	Pipeline de infraestructura
Dependencias permitidas entre servicios	AuthorizationPolicy generada desde el modelo C4 (as-code); llamada no declarada = denegada	Despliegue y runtime
Profundidad de cadena síncrona ≤ 3	Análisis de trazas en pruebas de carga y alerta en producción	Preproducción y runtime
Sin lógica de negocio en el gateway	Solo plugins de la lista permitida en la configuración declarativa	Pull request de configuración
Reglas de capas dentro del servicio	Pruebas ArchUnit: el dominio no depende de adaptadores ni de frameworks	Build
Metadatos de propiedad y observabilidad	Scorecard del catálogo: dueño, SLO, runbook, dashboards	Catálogo

13.4 Métricas de gobierno

Se siguen pocas métricas, pero con decisión asociada: porcentaje de APIs con contrato publicado antes de implementarse, número de cambios incompatibles por trimestre, tiempo de retiro de versiones obsoletas, servicios con scorecard completo y consumidores en versiones deprecadas. Si una métrica empeora dos meses seguidos, se revisa en la autoridad de diseño.

14. Plan de migración gradual

14.1 Enfoque

La migración se basa en tres ideas: primero se desacoplan los consumidores del core tradicional (sin mover datos), luego se lanzan productos nuevos en el core digital, y solo después se migran clientes existentes por cohortes pequeñas, con ejecución en paralelo y posibilidad de retorno. Cada fase entrega valor por sí misma, de modo que si el programa se detiene en cualquier punto, el Banco queda mejor que al inicio.

14.2 Fases

Tabla 34. Fases de la migración

Fase	Objetivo y alcance	Criterio de salida	Reversión	Duración indicativa
F0 · Fundaciones	Landing zone, enlaces dedicados, identidad, gateway, Kafka, observabilidad, pipelines y plantillas de servicio. CDC sobre el core tradicional en modo lectura.	Pruebas de DR de la plataforma superadas; CDC con lag p99 < 2 s durante 30 días.	No afecta producción.	4 meses
F1 · Fachada sobre el core tradicional	Legacy Core Adapter, Core Integration (una sola ruta), Customer Position, Party. Nuevos canales web y móvil sobre estas APIs; los canales actuales siguen operando.	Nuevos canales en producción para el 100 % de clientes; conciliación diaria sin diferencias.	Los canales anteriores siguen vivos como contingencia.	6 meses
F2 · Pagos, fraude y riesgos	Payment Order, Payment Execution, Fraud Screening y Limits. Migración de transferencias al nuevo flujo por tipo de operación.	Tasa de éxito ≥ a la del flujo anterior durante 60 días.	Feature flag por tipo de operación hacia el flujo anterior.	4 meses (paralelo a F1 en su tramo final)
F3 · Productos nuevos en core digital	Lanzamiento de ahorro digital en el core digital. Activación de la ruta DIGITAL por producto.	Producto estable; conciliación de cuentas puente en cero.	Suspender venta del producto; las cuentas existentes siguen operando.	3 meses
F4 · Open Finance	Consent Service, APIs externas, portal de desarrolladores, piloto con terceros seleccionados.	Certificación de conformidad FAPI; piloto sin incidentes de seguridad.	Revocación de credenciales del piloto.	4 meses
F5 · Migración de cartera por cohortes	Cohortes crecientes: colaboradores, clientes digitales con un solo producto, luego el resto. Dual-run antes de cada cohorte.	Cada cohorte: diferencias de conciliación en cero durante el periodo de observación.	Regla de ruteo de vuelta a LEGACY dentro de la ventana de retorno.	12 a 18 meses
F6 · Retiro	Apagado de canales antiguos e interfaces del core tradicional sin consumo; decisión sobre el core tradicional residual.	Consumo cero verificado por trazas durante 90 días.	No aplica.	Según cartera remanente

14.3 Migración de una cohorte

1. **Selección:** la cohorte se define por criterios de bajo riesgo (productos simples, sin créditos en mora, sin embargos judiciales). Se comunica a los clientes con anticipación.
2. **Dual-run:** durante dos semanas, el Reconciliation Service replica las operaciones de la cohorte en el core digital en modo sombra y compara saldos, intereses y comisiones con el core tradicional.
3. **Ventana de corte:** fuera de horario, se bloquean temporalmente las operaciones de la cohorte, se extraen saldos y datos, se cargan en el core digital y se concilian uno a uno.
4. **Cambio de SoR:** si la conciliación cuadra, se registra la regla de cohorte en el Routing Registry con fecha efectiva y aprobación dual. Se publica migration.cohort.migrated.v1.

5. **Observación:** durante 30 días se mantiene la posibilidad de retorno. Las cuentas en el core tradicional quedan bloqueadas, no cerradas.
6. **Cierre:** pasada la observación sin diferencias, se cierran las cuentas antiguas y se libera la cohorte siguiente.

> El canal no cambia en ningún paso. El cliente usa la misma app y el mismo identificador de cuenta; lo único que nota es la ventana de mantenimiento, que es corta.

14.4 Riesgos de la migración

Tabla 35. Riesgos principales y mitigación

Riesgo	Impacto	Mitigación
Diferencias de cálculo de intereses o comisiones entre cores	Alto	Dual-run por cohorte; parametrización validada por el área de productos antes de F5.
Capacidad insuficiente del core tradicional ante el tráfico de nuevos canales	Alto	Read models, bulkhead en el adaptador y pruebas de carga en F1.
Saldos residuales en cuentas puente	Medio	Conciliación intradía y alerta automática; procedimiento operativo definido.
Dependencia del proveedor del core digital	Medio	ACL y modelo canónico propios; cláusulas de salida y portabilidad de datos en el contrato.
Resistencia o fatiga de los equipos por la convivencia prolongada	Medio	Fases cortas con entregables visibles; plantillas y plataforma que reducen trabajo repetitivo.
Cambios regulatorios durante el programa (Open Finance)	Medio	APIs basadas en estándares internacionales y gateway que permite versionar sin tocar dominio.

15. Observabilidad y trazabilidad

15.1 Modelo de observabilidad

El modelo distingue cuatro tipos de información con propósitos y retenciones distintos. Mezclarlos es un error frecuente: los logs técnicos no sirven como evidencia regulatoria y la auditoría no sirve para depurar.

Tabla 36. Señales de observabilidad y su uso

Señal	Contenido	Herramienta	Retención
Trazas	Recorrido de cada solicitud entre servicios, incluyendo saltos asíncronos	OpenTelemetry → Tempo	15 días (100 % de errores y lentas; 20 % del resto)
Métricas	RED por servicio, USE por recurso, lag de consumidores, métricas de negocio	OpenTelemetry → Mimir/Prometheus	13 meses agregadas
Logs	Eventos técnicos estructurados en JSON, con trace_id y sin datos personales	OpenTelemetry → Loki	30 días en caliente, 1 año en frío
Auditoría	Quién hizo qué, sobre qué recurso, cuándo, desde dónde y con qué resultado	Audit Trail (WORM, encadenado por hash)	Según normativa bancaria

15.2 Propagación del contexto de punta a punta

- **HTTP:** el gateway crea o respeta traceparent y lo propaga; los BFF y servicios lo continúan automáticamente con la instrumentación de OpenTelemetry.
- Kafka: el outbox guarda el traceparent junto con el evento y el relay lo coloca como atributo CloudEvents y cabecera Kafka. El consumidor crea un span enlazado (span link), de modo que la traza de una transferencia incluye el procesamiento asíncrono en Payment Execution y Notification.
- MQ del core tradicional: el Legacy Core Adapter usa el identificador de traza como CorrelationId del mensaje y registra la correspondencia, lo que permite seguir una operación hasta el core.
- Core y rail: el paymentOrderId viaja como referencia externa al core y como referencia de extremo a extremo en ISO 20022 (EndToEndId), y vuelve en los eventos de CDC y en las respuestas del rail.

Tabla 37. Identificadores de correlación

Identificador	Lo genera	Viaja en	Sirve para
trace_id	API Gateway	traceparent en HTTP y Kafka; CorrelationId en MQ	Diagnóstico técnico de punta a punta.
paymentOrderId	Payment Order Service	APIs, eventos, referencia externa del core, EndToEndId ISO 20022	Trazabilidad de negocio y conciliación.
Idempotency-Key	Canal o tercero	Cabecera HTTP	Detectar reintentos del cliente.
x-fapi-interaction-id	Tercero	Cabecera Open Finance	Soporte y disputas con terceros.
consentId	Consent Service	Token interno, eventos	Demostrar con qué autorización se accedió a datos.

15.3 SLO y alertas

Las alertas se basan en consumo del presupuesto de error de los SLO (burn rate) y no en umbrales de CPU, para que una alerta signifique impacto real en clientes.

Tabla 38. SLO de los recorridos principales

Recorrido	Indicador (SLI)	Objetivo	Alerta
Inicio de sesión	Logins exitosos / intentos válidos	99,9 %	Burn rate 14× en 1 h
Consulta de posición	Respuestas < 300 ms y sin error	99,5 %	Burn rate 6× en 6 h
Aceptación de transferencia	Respuestas 202 < 800 ms / solicitudes válidas	99,5 %	Burn rate 14× en 1 h
Finalización de transferencia	Órdenes en estado final < 60 s	99 %	Órdenes pendientes > umbral
Frescura del read model	Lag entre commit en core y read model	p99 < 2 s	Lag > 10 s por 5 min
DLQ	Mensajes en DLQ de dominios N0/N1	0	Cualquier mensaje

15.4 Auditoría y trazabilidad regulatoria

El Audit Trail recibe registros de los servicios a través de un topic dedicado y los almacena en un repositorio de solo anexado con bloqueo de objetos en modo de cumplimiento (nadie, ni administradores, puede borrar antes de la retención). Cada registro incluye el hash del anterior, lo que permite demostrar que la secuencia no fue alterada. Se auditan las operaciones monetarias, los cambios de datos del cliente, los accesos a datos confidenciales (requisito de responsabilidad proactiva de la LOPDP), los cambios de consentimiento, los cambios en el Routing Registry y las acciones administrativas.

15.5 Tableros

- **Técnico:** salud por servicio, dependencias, lag de Kafka, estado de circuit breakers hacia cada core.
- **Negocio:** transferencias por minuto, tasa de rechazo por motivo, montos en espera, uso de APIs por tercero.
- **Migración:** clientes por core, diferencias de conciliación, saldo de cuentas puente.
- **Seguridad:** intentos de autenticación fallidos, step-up exigidos, denegaciones del mesh y del PEP, enviados también al SIEM.

16. Riesgos de la arquitectura y próximos pasos

Tabla 39. Decisiones abiertas y riesgos de diseño

Tema	Situación y propuesta
Capacidad real del core tradicional	Validar el supuesto S1 con pruebas de carga en F0. Si la capacidad es menor, ampliar el uso de read models y colas para operaciones diferibles.
Formato del rail interbancario	Confirmar con la plataforma de pagos los formatos vigentes; el diseño los absorbe en Payment Execution.
Norma local de Open Finance	Mantener las APIs alineadas a FAPI 2.0 y ajustar recursos y permisos cuando se publique la norma.
Región de nube y transferencia internacional	Formalizar con el Delegado de Protección de Datos la EIPD y las garantías contractuales antes de F0.
Capacidades del core digital	Confirmar soporte de retenciones, referencias externas idempotentes y eventos; si falta alguna, se refleja en los puertos implementados.

Como próximos pasos se propone: un taller de validación de supuestos con las áreas de tecnología, pagos y riesgos; una prueba de concepto de dos semanas del Legacy Core Adapter con CDC sobre una réplica del core tradicional, que es el componente de mayor incertidumbre técnica; y la elaboración de la EIPD de la plataforma.

Anexo A. Glosario

Término	Significado en este documento
ACL	Anti-Corruption Layer. Componente que traduce entre el modelo de un sistema externo y el modelo canónico.
BFF	Backend for Frontend. Servicio que adapta las APIs de dominio a un canal específico.
BIAN	Banking Industry Architecture Network. Estándar de referencia de capacidades y servicios bancarios.
CDC	Change Data Capture. Captura de cambios confirmados en una base de datos a partir de su log.
Cohorte	Grupo de clientes que se migra al core digital en una misma ventana.
DLQ	Dead Letter Queue. Destino de mensajes que no pudieron procesarse tras los reintentos.
DPoP	Demonstrating Proof of Possession. Mecanismo que liga un token a una llave del cliente.
FAPI 2.0	Perfil de seguridad de OAuth para APIs financieras de alto valor.
PAR / RAR	Pushed Authorization Requests (RFC 9126) / Rich Authorization Requests (RFC 9396).
Read model	Copia derivada y optimizada para consulta; nunca es sistema de registro.
SoR	System of Record. Sistema autoritativo de un dato.
SPIFFE	Estándar de identidad de cargas de trabajo usado por el service mesh.
Warm standby	Entorno secundario desplegado con capacidad mínima y datos replicados, listo para escalar.